TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN 1

ĐỀ TÀI: THIẾT KẾ PHẦN CỨNG TĂNG TỐC XỬ LÍ SPEECH-TO-SPEECH

Môn học: CE213.Q23 – Thiết kế hệ thống số với HDL

Giảng viên hướng dẫn: Phạm Hoài Luân

Thực hiện bởi nhóm 6, bao gồm:

1.	Nguyễn Đặng Phương Duy	23520372	Trưởng nhóm
2.	Vũ Đại Dương	23520359	Thành viên

Thời gian thực hiện: 14/03/2026 – 06/07/2026

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án môn học này, em đã nhận được nhiều sự hướng dẫn và hỗ trợ quý báu từ quý thầy cô và bạn bè.

Em xin gửi lời cảm ơn chân thành đến thầy Phạm Hoài Luân đã tận tình hướng dẫn, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những kiến thức và kinh nghiệm mà thầy chia sẻ đã giúp em hoàn thiện đồ án một cách tốt hơn.

Em cũng xin cảm ơn quý thầy cô trong khoa đã truyền đạt những kiến thức nền tảng trong suốt quá trình học tập, tạo điều kiện để em có thể thực hiện đề tài này.

Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn động viên và hỗ trợ em trong quá trình học tập cũng như hoàn thành đồ án.

Mặc dù đã cố gắng hoàn thiện trong khả năng cho phép, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ thầy để có thể tiếp tục hoàn thiện hơn trong tương lai.

MỤC LỤC

LỜI CẢM ƠN 2
MỤC LỤC 3
DANH SÁCH CHỮ VIẾT TẮT 6
DANH SÁCH HÌNH 8
DANH SÁCH BẢNG 9
PHÂN CÔNG CÔNG VIỆC 10
LỜI MỞ ĐẦU 11
Chương I. TỔNG QUAN. 12
1. Giới thiệu đề tài. 12
1.1. Bối cảnh và nội dung chính. 12
1.2. Lý do chọn đề tài. 14
1.3. Các môn học liên quan. 14
2. Cơ sở lý thuyết. 15
2.1. Triển khai LLM trên KIT AMD Kria KV260. 15
2.2. Mô hình ngôn ngữ Qwen2 và vai trò của phép nhân ma trận. 16
2.3. Lượng tử hóa đối xứng. 16
2.4. Thuật toán nhân ma trận phân khối. 17
2.5. Kiến trúc mảng tâm thu và luồng dữ liệu Output Stationary. 18
2.6. Kỹ thuật đệm Ping-Pong trên BRAM. 20
2.7. Lõi truyền dữ liệu AXI-Datamover. 21
2.8. Giao thức giao tiếp Bus họ AMBA AXI4 21
2.9. Tái chuẩn hóa và đánh giá 23
Chương II. KIẾN TRÚC HỆ THỐNG 25
Chương III. THIẾT KẾ HỆ THỐNG PHẦN CỨNG. 27
1. Thiết kế bộ tăng tốc nhân ma trận. 27
1.1. Module l2_tiling_agu – Điều phối luồng dữ liệu cấp khối 27
1.1.1. Bộ đệm on-chip: GBUFA, GBUFB và C_ACCUM 28
1.1.2. Chuỗi trạng thái FSM Compute 28
1.2. Module gemm_l2_axi_lite_regs — Giao tiếp điều khiển từ CPU 29

1.3. Module gemm_compute_core — Lõi tính toán cấp tile.....30
1.4. Module bram_input_buffer — Đệm đầu vào ping-pong.....30
1.5. Module systolic_dataflow_ctrl — Bộ điều khiển luồng mảng tâm thu 30
1.6. Module skew_network — Mạng làm lệch dữ liệu 32
1.7. Lõi tính toán: mac_core và pe_wrapper 33
1.7.1. mac_core — Tối ưu cho DSP48E2 33
1.7.2. pe_wrapper — Phần tử xử lý với luồng drain 33
1.8. Module systolic_array_os — Mảng tâm thu 16×16 34
1.9. Module output_serializer — Bộ tuần tự hóa đầu ra 34
1.10. Module sc_fifo_fwft — Bộ đệm FIFO FWFT 35
1.11. Module post_accumulator — Tích lũy đa tile theo chiều K.....35
1.12. Module int32_int8_rescale — Tái chuẩn hóa lượng tử.....36
2. Triển khai hệ thống. 37
2.1. Kiến trúc giao tiếp 37
2.2. Luồng thực thi tổng thể 37
3. Kiểm tra và phân tích.....38
3.1. Kiểm tra chức năng.....38
3.2. Phân tích PPA của hệ thống 42
Chương IV. THỰC NGHIỆM HỆ THỐNG TRÊN FPGA. 47
1. Trích xuất ma trận đầu vào và đánh giá dữ liệu lượng tử hóa. 47
2. Triển khai phần cứng lên KV260 48
3. Phần mềm điều khiển 48
3.1. Ánh xạ bộ nhớ vật lý 48
3.2. Cấu hình thanh ghi và khởi động.....49
3.3. Cơ chế polling và đồng bộ hóa 49
3.4. Mô hình tham chiếu phần mềm 50
4. Kết quả thực nghiệm.....50
3.1. Kiểm thử chức năng — GEMM 512×512×512 50
3.2. Benchmark hiệu năng — GEMM 5120×1280×5120 50
Chương V. ĐÁNH GIÁ HỆ THỐNG. 52
1. Đánh giá hiệu năng xử lí giữa PS và bộ tăng tốc PL trên KV260 52

2. Đánh giá tính đúng đắn và hiệu năng với Python 52
2.1. Xác thực chéo logic phần cứng và độ chính xác Bit-True 52
2.2. Phân tích sai số và chất lượng lượng tử hóa cấu trúc LLM 53
Chương VI. KẾT LUẬN. 55
PHỤ LỤC 56
NGUỒN THAM KHẢO 57

DANH SÁCH CHỮ VIẾT TẮT

STT	Từ viết tắt	Ý nghĩa
1	AI	Artificial Intelligence
2	AXI	Advanced eXtensible Interface
3	BRAM	Block Random Access Memory
4	CPU	Central Processing Unit
5	DMA	Direct Memory Access
6	DSP	Digital Signal Processing
7	FFN	Feed-Forward Network
8	FIFO	First In First Out
9	FPGA	Field-Programmable Gate Array
10	FSM	Finite State Machine
11	FWFT	First-Word Fall-Through
12	GEMM	General Matrix Multiply
13	GPU	Graphic Processing Unit
14	HDL	Hardware Description Language
15	LLM	Large Language Model
16	MAC	Multiply-Accumulate
17	MatMul	Matrix Multiplication
18	MM	Memory-Mapped
19	OS	Output Stationary
20	PEs	Processing Elements
21	PL	Programmable Logic
22	PS	Processing System

23	SA	Systolic Array
24	SIMD	Single Instruction, Multiple Data
25	STA	Static Timing Analysis

DANH SÁCH HÌNH

Hình 1. Kết quả phân tích tỷ trọng ma trận của các lớp có trong Qwen2 12
Hình 2. KIT AMD Kria KV260 15
Hình 3. Tổng quan về thực hiện Tiling trên kiến trúc GEMM [12] 18
Hình 4. Kiến trúc tương quan của Systolic Array 19
Hình 5. Sơ đồ tổng quan của Ping-pong Buffer [17] 20
Hình 6. Tổng quan hệ thống 25
Hình 7. FSM của module l2_tiling_agu 28
Hình 8. FSM của module systolic_dataflow_ctrl 31
Hình 9. Minh họa skew network cho ma trận 4x4 32
Hình 10. Thiết kế cụ thể cho module pe_wrapper 33
Hình 11. FSM của module output_serializer 34
Hình 12. Hệ thống chi tiết được triển khai trong Vivado 37
Hình 13. Waveform ghi dữ liệu từ AXI DMA vào BRAM buffer 40
Hình 14. Waveform skew dữ liệu nhận từ BRAM 41
Hình 15. Waveform tín hiệu điều khiển SA và lưu kết quả vào FIFO 41
Hình 16. Waveform tái chuẩn hóa dữ liệu và truyền AXI-Stream output 42
Hình 17. Phân bố tài nguyên theo từng IP/module trong hệ thống 44
Hình 18. Kết quả phân tích Total On-chip Power sau Implementation 45
Hình 19. Kiểm tra kích thước ma trận đã trích xuất 47
Hình 20. Kết quả đối chiếu chất lượng lượng tử hóa đầu vào 47
Hình 21. Kết quả xử lý 512×512×512 trên KV260 50
Hình 22. Kết quả benchmark 5120×1280×5120 trên KV260 51
Hình 23. Kết quả kiểm tra Bit-True 53
Hình 24. Kết quả đánh giá giải lượng tử kết quả GEMM 53

DANH SÁCH BẢNG

Bảng 1. So sánh thời gian tính toán giữa phương pháp thông thường và Systolic Array 20
Bảng 2. Phân tích FSM Compute của l2_tiling_agu 29
Bảng 3. Phân tích FSM systolic_dataflow_ctrl 32
Bảng 4. Phân tích FSM output_serializer 35
Bảng 5. Phân tích FSM post_accumulator 36
Bảng 6. Kết quả hiển thị trên Console 40
Bảng 7. Kết quả phân tích timing sau Implementation 43
Bảng 8. Mức sử dụng tài nguyên FPGA sau Implementation 43
Bảng 9. Ánh xạ bộ nhớ vật lý trong chương trình điều khiển 48
Bảng 10. Danh sách thanh ghi cấu hình AXI4-Lite 49

PHÂN CÔNG CÔNG VIỆC

Thành viên	Công việc	Đóng góp (%)
Nguyễn Đặng Phương Duy	- Phân tích model Qwen2 - Thiết kế ping-pong buffer - Thiết kế firmware - Nạp kiểm thử trên FPGA	50%
Vũ Đại Dương	- Thiết kế các module còn lại - Thực hiện phân tích bằng code Python - Viết báo cáo	50%

LỜI MỞ ĐẦU

Trong các mô hình trí tuệ nhân tạo hiện đại, đặc biệt là Large Language Model (LLM), phép nhân ma trận tổng quát (General Matrix Multiplication – GEMM) chiếm phần lớn khối lượng tính toán và băng thông bộ nhớ. Khi kích thước mô hình ngày càng tăng, việc xử lý bằng CPU hoặc GPU trên các thiết bị biên gặp nhiều hạn chế về hiệu năng, điện năng tiêu thụ và chi phí triển khai. Vì vậy, FPGA trở thành một nền tảng phù hợp nhờ khả năng xử lý song song, khả năng tái cấu hình và hiệu năng trên mỗi watt cao.

Xuất phát từ yêu cầu đó, đồ án tập trung nghiên cứu, thiết kế và hiện thực bộ tăng tốc GEMM hoàn chỉnh trên nền tảng AMD Kria KV260. Hệ thống sử dụng kiến trúc Systolic Array 16×16 theo cơ chế Output Stationary, kết hợp tính toán INT8 nhằm tăng thông lượng và giảm băng thông bộ nhớ. Bên cạnh khối tính toán, thiết kế còn tích hợp module cắt ma trận gốc về kích thước 256x256 thực hiện hoàn toàn bằng phần cứng, giao tiếp cấu hình thông qua AXI4-Lite, truyền dữ liệu bằng AXI DataMover kết hợp AXI4-Stream, hệ thống đệm Ping-Pong BRAM, FIFO FWFT, khối cộng dồn và Rescale INT32 về INT8. Ngoài ra, các tín hiệu ngắt của hệ thống được tổng hợp thông qua xlconcat để đồng bộ với bộ xử lý ARM, tạo thành một hệ thống gia tốc hoàn chỉnh từ cấu hình, truyền dữ liệu, xử lý đến ghi trả kết quả.

Hệ thống được hiện thực và kiểm thử trên board AMD Kria KV260 với nhiều kích thước ma trận khác nhau. Kết quả thực nghiệm cho thấy bộ gia tốc đạt tốc độ xử lý vượt trội so với xử lý thuần trên CPU ARM, đồng thời cho kết quả đầu ra hoàn toàn trùng khớp với mô hình tham chiếu trên Python. Kết quả này khẳng định tính khả thi của việc triển khai các phép toán GEMM hiệu năng cao trên FPGA, tạo tiền đề cho việc xây dựng các bộ gia tốc phục vụ các ứng dụng AI và LLM trên thiết bị biên.

Chương I. TỔNG QUAN.

1. Giới thiệu đề tài.

1.1. Bối cảnh và nội dung chính.

Sự phát triển mạnh mẽ của các mô hình ngôn ngữ lớn (LLM) và mô hình đa phương thức quy mô lớn, tiêu biểu là họ mô hình Qwen2 với dải tham số trải dài từ 0.5 đến 72 tỷ tham số, đã mang lại những bước đột phá vượt bậc trong xử lý ngôn ngữ tự nhiên và tín hiệu âm thanh phức tạp [1]. Kiến trúc cốt lõi cấu thành nên sự thành công của các hệ thống này là mạng Transformer, một mô hình xử lý song song dựa hoàn toàn trên cơ chế Self-Attention. Tuy nhiên, để đổi lấy năng lực biểu diễn ngữ cảnh mạnh mẽ, Transformer đòi hỏi một khối lượng tài nguyên tính toán khổng lồ cùng băng thông bộ nhớ cực lớn, tạo ra thách thức nghiêm trọng khi triển khai trên các thiết bị thực tế [2].

```
🎧 Đang tạo tín hiệu Audio 55 giây (Stress Test NPU)...
🚀 Đang chạy mô hình (với âm thanh 55s) để đo đạc từng Layer...

========== BÁO CÁO PHÂN TÍCH TỶ TRỌNG (STRESS TEST 55s) ==========
⏱️ Tổng thời gian chạy toàn bộ mô hình: 2.3720 giây
----------------------------------------------------------------------
1. Attention - Q Proj (q_proj)               : 0.1044 s  |  Chiếm:  4.40%
2. Attention - K Proj (k_proj)               : 0.0466 s  |  Chiếm:  1.97%
3. Attention - V Proj (v_proj)               : 0.0416 s  |  Chiếm:  1.75%
4. Attention - O Proj (o_proj)               : 0.0073 s  |  Chiếm:  0.31%
5. Mạng FFN (gate, up, down_proj, fc1, fc2)  : 0.4122 s  |  Chiếm: 17.38%
6. Audio Encoder (Conv1d, Conv2d)            : 0.5601 s  |  Chiếm: 23.61%
7. Attention Core (QxK, Softmax, SV)         : 0.4663 s  |  Chiếm: 19.66%
8. Các lớp Linear phân loại khác             : 0.0843 s  |  Chiếm:  3.55%
----------------------------------------------------------------------
TỔNG TỶ TRỌNG CÁC LỚP MA TRẬN THUẦN TÚY (Thuần MatMul): 29.36%
   (Thời gian thực thi: 0.6964 giây)
======================================================================
```

Hình 1. Kết quả phân tích tỷ trọng ma trận của các lớp có trong Qwen2

Để có số liệu thực nghiệm làm cơ sở khoa học, nhóm nghiên cứu đã tiến hành chạy thử nghiệm stress test trực tiếp trên mô hình tinh chỉnh *Qwen2-Audio-Dialogue* được cấu hình chạy ở chế độ chính xác cao nhất FP32 để bắt giữ toàn bộ các giá trị ngoại lai (outliers) [3]. Với đầu vào là một đoạn tín hiệu âm thanh thực tế kéo dài 55 giây nhằm ép hệ thống hoạt động ở ngưỡng tải tối đa, kết quả phân tích tỷ trọng cho thấy các phép toán nhân ma trận mật độ cao (Dense Matrix Multiplication - GEMM)—bao gồm các lớp chiếu tuyến tính *Q, K, V, O* của bộ lõi Attention và các lớp phân tách *gate_proj, up_proj, down_proj* trong mạng khuếch đại Feed-Forward Network (FFN)—chiếm tỷ trọng áp đảo lên tới hơn 29.36% tổng thời gian thực thi của mô hình. Điều này

chứng minh toán tử nhân ma trận chính là nút thắt cổ chai cục bộ làm suy giảm hiệu năng xử lý của toàn bộ hệ thống.

Đề tài tập trung nghiên cứu, thiết kế và hiện thực hóa một bộ tăng tốc phần cứng chuyên dụng trên nền tảng FPGA AMD Kria KV260 nhằm tăng tốc phép nhân ma trận là phép toán chiếm phần lớn khối lượng tính toán trong các mô hình AI và LLM. Quá trình nghiên cứu và phát triển hệ thống được thực hiện qua sáu giai đoạn chính như sau:

1. Trích xuất và lượng tử hóa dữ liệu: Trích xuất các ma trận trọng số (Weights) và kích hoạt (Activations) đã được lượng tử hóa INT8 từ mô hình Qwen2 để làm dữ liệu đầu vào cho bộ gia tốc, đồng thời xác định các tham số Scale và Zero-point phục vụ quá trình tái chuẩn hóa kết quả trên phần cứng [4].

2. Thiết kế khối tính toán phần cứng: Xây dựng kiến trúc Systolic Array 16×16 theo cơ chế Output Stationary, kết hợp hệ thống Ping-Pong BRAM Buffer, mạng skew dữ liệu, bộ điều khiển pipeline và bộ tuần tự hóa đầu ra nhằm thực hiện phép nhân ma trận song song với thông lượng cao.

3. Xây dựng hệ thống gia tốc hoàn chỉnh: Tích hợp Matrix Tiling Controller thực hiện phân khối ma trận kích thước 256×256 hoàn toàn bằng phần cứng, sử dụng giao tiếp AXI4-Lite để cấu hình hệ thống, IP AXI DataMover kết hợp giao thức AXI4-Stream để truyền dữ liệu giữa DDR và bộ gia tốc, đồng thời sử dụng IP xlconcat để tổng hợp các tín hiệu ngắt phục vụ quá trình điều khiển giữa PS và PL.

4. Xử lý và tái chuẩn hóa kết quả: Thiết kế khối cộng dồn, FIFO First-Word Fall-Through (FWFT) và module Rescale nhằm chuyển đổi kết quả tích lũy từ INT32 về INT8 theo các tham số lượng tử hóa, trước khi ghi trở lại bộ nhớ DDR để phục vụ các giai đoạn suy luận tiếp theo.

5. Triển khai hệ thống trên phần cứng: Hiện thực thiết kế trên board AMD Kria KV260, xây dựng chương trình điều khiển trên ARM Cortex-A53 để cấu hình hệ thống, điều khiển IP AXI DataMover và phối hợp hoạt động giữa PS và PL trong quá trình xử lý các bài toán GEMM với nhiều kích thước ma trận khác nhau.

6. Đánh giá và kiểm chứng hệ thống: Đối chiếu kết quả đầu ra INT8 của bộ gia tốc với Golden Model trên Python, đồng thời đánh giá độ chính xác thông qua các chỉ số Cosine Similarity, Signal-to-Quantization-Noise Ratio (SQNR), Root Mean Square Error (RMSE) và Mean Absolute Error (MAE). Bên cạnh đó, tiến hành so sánh hiệu năng giữa xử lý thuần trên CPU và bộ tăng tốc phần cứng để chứng minh khả năng tăng tốc cũng như bảo toàn đặc trưng của dữ liệu sau lượng tử hóa.

1.2. Lý do chọn đề tài.

Nhóm nghiên cứu quyết định lựa chọn đề tài này xuất phát từ 4 động lực khoa học và thực tiễn cốt lõi sau:

Vượt qua Memory Wall: Theo lý thuyết phân tích hiệu năng của mô hình Roofline Model, các kiến trúc máy tính truyền thống (Von Neumann) khi xử lý các mô hình AI quy mô lớn thường bị giới hạn nghiêm trọng bởi băng thông truy xuất bộ nhớ ngoài trước khi chạm được tới ngưỡng giới hạn năng lực tính toán lý thuyết của chip [5]. Việc thiết kế một kiến trúc phần cứng chuyên dụng cho phép phân cấp và tái sử dụng dữ liệu triệt để ngay trên chip nhằm phá vỡ giới hạn này.

Tối ưu hóa tài nguyên nhờ lượng tử hóa số nguyên: Các nghiên cứu thực nghiệm quy mô lớn của Google trên bộ xử lý TPU (Tensor Processing Unit) đã chứng minh rằng việc thay thế các đường dẫn tính toán dấu phẩy động bằng mạch toán tử INT8 giúp giảm tới 6 lần năng lượng tiêu thụ và diện tích vi khối silicon, trong khi cho phép tích hợp mật độ bộ nhân cộng (MAC) cao gấp hàng chục lần trên cùng một đơn vị diện tích phần cứng [6].

Ưu thế kiến trúc của Mảng tâm thu: Đối với các toán tử nhân ma trận trù mật, kiến trúc SA chứng minh sự vượt trội hoàn toàn so với kiến trúc SIMD (Single Instruction, Multiple Data) truyền thống trên GPU nhờ cơ chế truyền tải và chia sẻ dữ liệu trực tiếp giữa các phần tử xử lý lân cận (Processing Elements - PEs), giảm thiểu việc truy xuất vào tập thanh ghi trung tâm, từ đó triệt tiêu độ trễ bus và tiết kiệm điện năng [7].

Thúc đẩy xu hướng Edge AI: Việc hiện thực hóa thành công bộ gia tốc GEMM trên các dòng Edge FPGA cấu hình thấp chứng minh tính khả thi của việc đưa các mô hình trí tuệ nhân tạo phức tạp ra vận hành tại các thiết bị đầu cuối với không gian tài nguyên hạn chế và nghiêm ngặt về công suất tiêu thụ [8].

1.3. Các môn học liên quan.

Đề tài là kết quả của sự vận dụng tổng hợp và đào sâu kiến thức nền tảng từ bốn phân nhánh môn học chuyên ngành cốt lõi:

Kiến trúc máy tính & Vi xử lý: Cung cấp cơ sở lý thuyết về tổ chức hệ thống, cấu trúc phân cấp bộ nhớ, phân tích xung đột đường ống, cơ chế truy cập bộ nhớ trực tiếp và mô hình trao đổi dữ liệu qua hệ thống Bus ngoại vi.

Thiết kế hệ thống số với HDL: Nền tảng kỹ thuật toán học để mô tả cấu trúc phần cứng bằng ngôn ngữ cấu trúc Verilog, thiết kế máy trạng thái mạch logic (FSM), tối ưu hóa mạch tổ hợp và mạch tuần tự, phân tích Timing (Setup/Hold time) và kiểm soát phân bố tài nguyên trên vi mạch FPGA [9].

Thiết kế hệ thống nhúng: Phương pháp luận phối hợp thiết kế đồng thời Phần cứng/Phần mềm, kỹ thuật viết driver điều khiển phần cứng bằng ngôn ngữ C/C++ cấp thấp, cấu hình hệ điều hành Linux nhúng và quản lý bộ nhớ vật lý.

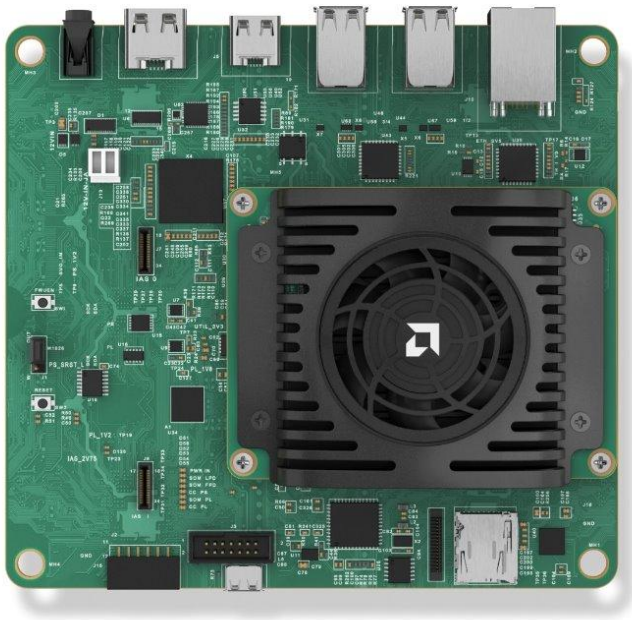
Trí tuệ nhân tạo & Học máy: Cung cấp hiểu biết toán học về cấu trúc mạng nơ-ron, cơ chế Forward Pass, các hàm kích hoạt và nguyên lý tối ưu hóa hàm toán học thông qua các giải pháp thu gọn biểu diễn số liệu.

2. Cơ sở lý thuyết.

Để hiện thực hóa hệ thống gia tốc trên, đề tài áp dụng các lý thuyết và công nghệ cốt lõi sau:

2.1. Triển khai LLM trên KIT AMD Kria KV260.

Việc triển khai các nguyên mẫu LLM xuống phần cứng biên đang là xu hướng tất yếu nhằm giảm độ trễ và tăng hiệu năng xử lý tại thiết bị đầu cuối [8]. Trong đồ án này, nhóm lựa chọn AMD Kria KV260 Vision AI Starter Kit làm nền tảng triển khai bộ gia tốc GEMM. KV260 sử dụng SoC Zynq UltraScale+ MPSoC, tích hợp đồng thời hai miền xử lý gồm Processing System (PS) và Programmable Logic (PL) [10], [11]. Kiến trúc này đặc biệt phù hợp với bài toán tăng tốc AI khi cho phép kết hợp xử lý phần mềm linh hoạt với khả năng xử lý song song của phần cứng FPGA.



Hình 2. KIT AMD Kria KV260

Về phía PS, KV260 sử dụng bộ vi xử lý lõi tứ ARM Cortex-A53 64-bit, đồng thời tích hợp các ngoại vi như bộ điều khiển DDR4, Ethernet, USB và AXI Interconnect. Board được trang bị 4 GB DDR4 RAM, đủ khả năng lưu trữ các ma trận kích thước lớn

phục vụ quá trình kiểm thử. Với dữ liệu INT8, ma trận kích thước 5120×1280 chỉ chiếm khoảng 6.25 MB bộ nhớ, trong khi ma trận đầu ra 5120×5120 chiếm khoảng 25 MB, hoàn toàn nằm trong khả năng lưu trữ của hệ thống DDR4 trên KV260.

Về phía PL, thiết bị FPGA trên KV260 cung cấp khoảng 256K LUTs, 512K Flip-Flops, 144 Block RAM (BRAM) và 1248 khối DSP48E2 [10]. Đây là các tài nguyên quan trọng phục vụ xây dựng các kiến trúc xử lý song song cho bài toán GEMM.

Sự kết hợp giữa PS và PL giúp hệ thống tận dụng được tính linh hoạt của phần mềm và khả năng xử lý song song hiệu năng cao của FPGA, từ đó phù hợp với các bài toán GEMM kích thước lớn trong AI và LLM.

2.2. Mô hình ngôn ngữ Qwen2 và vai trò của phép nhân ma trận.

Mô hình toán học cơ sở của họ mô hình Qwen2 hoạt động dựa trên cơ chế Multi-Head Attention [1]. Tại mỗi lớp Transformer, dữ liệu đầu vào kích hoạt X được nhân với các ma trận trọng số tương ứng W_Q, W_K, W_V nhằm tạo ra ba ma trận đặc trưng: Truy vấn (Query - Q), Khóa (Key - K), và Giá trị (Value - V) theo phương trình [2]:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V$$

Sau đó, toán tử Self-Attention lõi được thực thi bằng cách tính toán độ tương đồng tích vô hướng giữa các vector đặc trưng, chuẩn hóa thông qua hàm Softmax và nhân tích lũy với ma trận Giá trị V [2]:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

Toàn bộ các pha biến đổi tuyến tính này, kết hợp với các toán tử chiếu lớp FFN sau đó ($FFN(x) = \text{Silu}(xW_{gate}) \cdot xW_{up} \cdot W_{down}$), bản chất đều là chuỗi lặp lại liên tục của các phép nhân ma trận dày đặc (GEMM). Do đó, tăng tốc độ thực thi phần cứng của toán tử MatMul sẽ trực tiếp quyết định thông lượng suy luận (Tokens/giây) và giảm thiểu tối đa độ trễ phản hồi của mô hình Qwen2 [1], [11].

2.3. Lượng tử hóa đối xứng.

Để tối ưu hóa diện tích vi mạch và giảm thiểu tối đa băng thông truyền tải dữ liệu giữa PS và PL, đề tài áp dụng giải pháp lượng tử hóa tuyến tính đối xứng tuyệt đối (Linear Symmetric Quantization) nhằm chuyển đổi dữ liệu từ miền số thực FP32 sang miền số nguyên giới hạn INT8 [4]. Thuật toán ánh xạ một không gian phân bố số thực liên tục $[-\alpha, \alpha]$ vào dải số nguyên đối xứng $[-127,127]$ với điểm $Zero\text{-}Point$ của hệ thực trùng khớp tuyệt đối với giá trị 0 của hệ nguyên [4]:

$$X_{\text{int8}} = \text{clamp}\left(\left\lfloor\frac{X_{\text{fp32}}}{\text{Scale}}\right\rceil, -128,127\right)$$

Trong đó, hệ số thu phóng (Scale Factor) được xác định dựa trên giá trị biên độ tuyệt đối cực đại của tensor dữ liệu nhằm bảo toàn tối đa Outliers có ý nghĩa quyết định đến chất lượng suy luận của LLM [4]:

$$\text{Scale} = \frac{\max(|X_{\text{fp32}}|)}{127}$$

Ưu điểm cơ bản của lượng tử hóa đối xứng khi triển khai phần cứng RTL là triệt tiêu hoàn toàn sự xuất hiện của tham số $Zero\text{-}Point$ ($Z_x = 0$). Trong các cấu trúc lượng tử hóa phi đối xứng (Asymmetric), phép nhân hai phần tử lượng tử hóa đòi hỏi mạch logic phải gánh thêm các bộ cộng đại số phức tạp để bù trừ khoảng lệch toán học:

$$(X_q - Z_x) \times (Y_q - Z_y) = X_q Y_q - X_q Z_y - Y_q Z_x + Z_x Z_y$$

Với lượng tử hóa đối xứng, phương trình toán học tại lõi nhân của cấu trúc PE rút gọn chỉ còn duy nhất một toán tử nhân số nguyên thuần túy ($X_q \times Y_q$). Phép giản lược này giúp tiết kiệm hàng ngàn cổng logic cấu thành bộ cộng, giảm độ trễ đường truyền tới hạn (Critical Path Delay), và cho phép mạch hoạt động ở tần số xung nhịp cao hơn [4], [7].

Sau khi mảng phần cứng thực hiện phép nhân cộng tích lũy, dữ liệu trung gian ngõ ra sẽ mang định dạng số nguyên 32-bit (INT32) để ngăn chặn hiện tượng tràn số. Giá trị này cần được tái chuẩn hóa về lại định dạng INT8 thông qua tích hiệu dụng của hai hệ số scale đầu vào cùng phép dịch bit phần cứng trước khi ghi ngược lại bộ nhớ hệ thống [4]:

$$C_{\text{int8}} = \text{clamp}\big((C_{\text{int32}} \times \text{scale_effective}) \gg \text{SCALE_SHIFT}, -128, 127\big)$$

Để đánh giá độ suy hao độ chính sắc sau quá trình lượng tử hóa phần cứng, bốn chỉ số đo lường chuẩn công nghiệp được áp dụng:

Cosine Similarity: Đo lường độ trùng khớp về mặt hướng của vector kết quả giữa phần cứng và phần mềm.

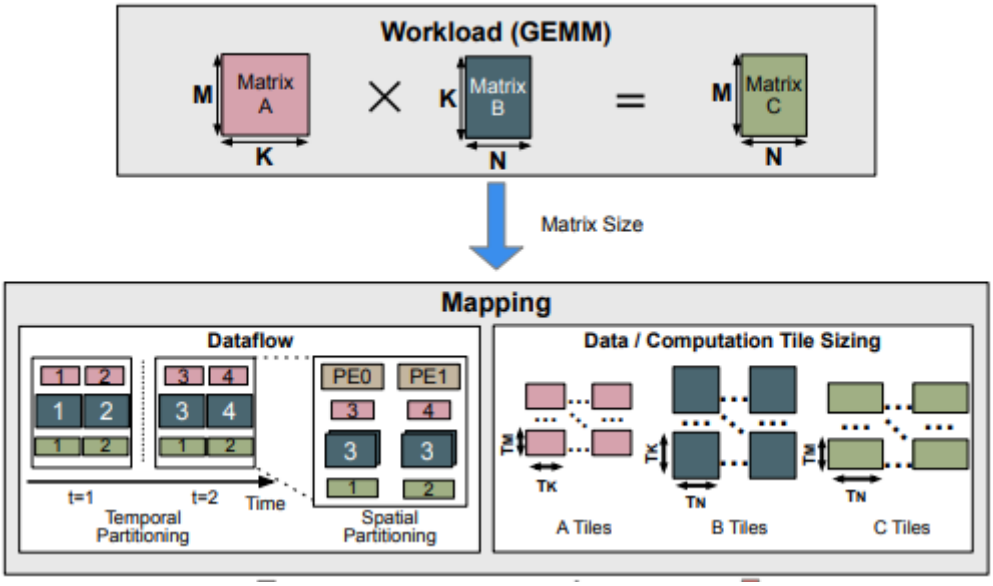
SQNR (Signal-to-Quantization-Noise Ratio): Tỷ số công suất tín hiệu trên công suất nhiễu lượng tử, định lượng mức độ méo dạng tín hiệu.

MAE (Mean Absolute Error) & RMSE (Root Mean Squared Error): Định lượng biên độ sai lệch tuyệt đối trung bình giữa giá trị thực và giá trị lượng tử hóa.

2.4. Thuật toán nhân ma trận phân khối.

Các kiến trúc Edge FPGA có dung lượng bộ nhớ nội vi on-chip (SRAM/BRAM) rất giới hạn, hoàn toàn không đủ không gian vật lý để lưu trữ đồng thời toàn bộ các ma

trận kích thước khổng lồ của mô hình Qwen2 [8], [11]. Do đó, việc áp dụng thuật toán chia khối dữ liệu là bắt buộc [12].



Hình 3. Tổng quan về thực hiện Tiling trên kiến trúc GEMM [12]

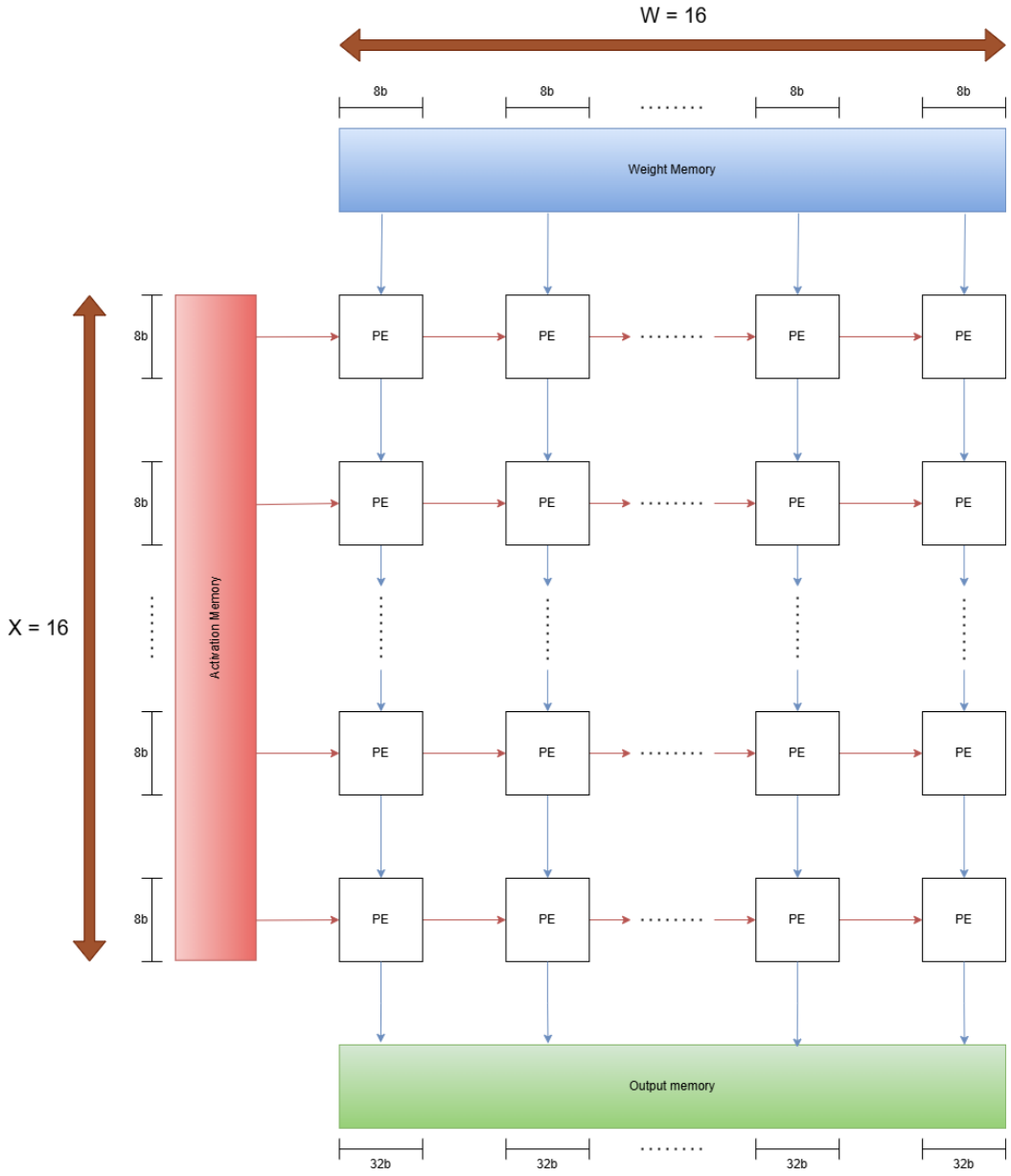
Thuật toán thực hiện phân rã phép nhân ma trận tổng thể $C = A \cdot B$ thành chuỗi các phép nhân ma trận con có kích thước cố định lớp vỏ ngoài khớp với cấu trúc phần cứng là *16 x 16* [12]. Để tăng cường thông lượng truy cập bộ nhớ, các khối BRAM lưu trữ dữ liệu này trên PL cần được cấu hình dưới dạng Dual-port RAM, cho phép một cổng liên tục nhận dữ liệu nạp từ bus ngoại vi trong khi cổng còn lại cung cấp dữ liệu tức thời cho mảng tính toán [17]. Đồng thời, áp dụng chỉ thị phân mảnh mảngtrên thiết kế để tách một khối bộ nhớ logic lớn thành nhiều block vật lý song song, cho phép mảng PE truy xuất đồng thời toàn bộ các phần tử trong cùng một hàng hoặc cột tại mỗi chu kỳ xung nhịp, triệt tiêu hoàn toàn độ trễ khởi tạo dữ liệu [9].

2.5. Kiến trúc mảng tâm thu và luồng dữ liệu Output Stationary.

Kiến trúc mảng tâm thu bao gồm một mạng lưới 2D gồm các phần tử xử lý PE cấu trúc đồng nhất, kết nối cục bộ với nhau theo mô hình lưới [13]. Dữ liệu từ bộ đệm sẽ truyền qua các PE một cách tuần tuần tự và đồng bộ theo từng nhịp xung nhịp, cho phép một phần tử dữ liệu sau khi đọc ra từ bộ nhớ được tái sử dụng nhiều lần tại nhiều PE khác nhau, giải quyết triệt để bài toán nghẽn cổ chai băng thông truyền tải dữ liệu của kiến trúc Von Neumann [7], [13].

Đề tài lựa chọn kiến trúc luồng dữ liệu OS cho mảng tâm thu kích thước *16 x 16* [12]. Trong kiến trúc này, mỗi phần tử PE tại tọa độ (i, j) bên trong mảng sẽ tích hợp một thanh ghi tích lũy nội vi đảm nhiệm việc lưu giữ cố định giá trị tổng riêng phần của phần tử ma trận kết quả $C[i][j]$ [13]. Tại mỗi chu kỳ xung nhịp, phần tử của ma trận kích hoạt A chảy xuyên qua mảng theo chiều ngang (từ trái sang phải), trong khi phần

tử của ma trận trọng số *B* chảy xuyên qua mảng theo chiều dọc (từ trên xuống dưới) [12], [13]. Các PE thực hiện phép nhân cộng (MAC) giữa hai giá trị nhận được và tích lũy trực tiếp vào thanh ghi tại chỗ [13].



Hình 4. Kiến trúc tương quan của Systolic Array

Ví dụ với phép nhân ma trận 2x2:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} x \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} PE0 & PE1 \\ PE2 & PE3 \end{bmatrix}$$

Bảng dưới đây thể hiện sự khác biệt giữa 2 phương pháp

Chu kì	Nhân ma trận thông thường	Systolic Array
1	PE0 = A x E	PE0 = A x E

2	PE0 = A x E + B x G	PE0 = A x E + B x G PE1 = A x F PE2 = C x E
3	PE1 = A x F	PE1 = A x F + B x H PE2 = C x E + D x G PE3 = C x F
4	PE1 = A x F + B x H	PE3 = C x F + D x H

Bảng 1. So sánh thời gian tính toán giữa phương pháp thông thường và Systolic Array

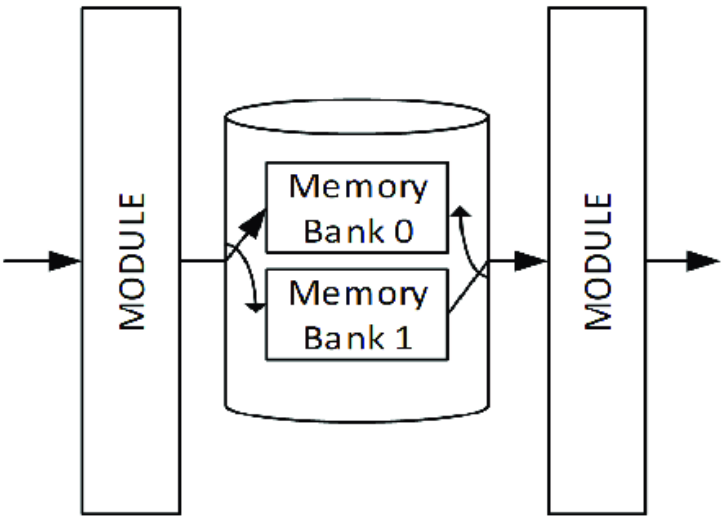
Có thể thấy rằng thuật toán thông thường cần xử lý tuần tự từng phần tử đầu ra, trong khi Systolic Array cho phép nhiều PE thực hiện phép MAC đồng thời. Nhờ đó, SA tận dụng tốt tính song song của phần cứng FPGA, giảm số chu kỳ xử lý và tăng thông lượng hệ thống.

So với kiến trúc SIMD truyền thống trên GPU - vốn yêu cầu đọc/ghi liên tục với hệ thống thanh ghi tập trung, Systolic Array chứng minh sự vượt trội khi xử lý Dense MatMul nhờ khả năng chuyển tiếp dữ liệu cục bộ giữa các PEs mà không cần truy xuất bộ nhớ ngoài [7], [11].

So với kiến trúc Weight Stationary (Cố định trọng số) đòi hỏi phải tích hợp thêm mạch điều khiển nạp trước trọng số và quản lý trạng thái phức tạp, kiến trúc OS cho phép ma trận Weight và Activation được đẩy vào mảng đồng bộ tuyến tính thông qua các cấu trúc thanh ghi dịch đơn giản, tối ưu đường truyền tín hiệu tần số cao [12], [13].

2.6. Kỹ thuật đệm Ping-Pong trên BRAM.

Nhằm ngăn chặn hiện tượng mảng tâm thu rơi vào trạng thái ngừng hoạt động chờ dữ liệu do độ trễ truyền tải của bus ngoại vi, kỹ thuật đệm kép Ping-Pong (Double Buffering) được triển khai trên hệ thống tài nguyên BRAM của phần PL [9], [14].



Hình 5. Sơ đồ tổng quan của Ping-pong Buffer [17]

Cơ chế vận hành của bộ đệm Ping-Pong chia bộ nhớ đệm thành hai phân vùng logic độc lập có kích thước bằng nhau: Phân vùng PING và phân vùng PONG [9]. Mạch điều khiển hoạt động dựa trên nguyên lý tương phản chéo chu kỳ:

Tại chu kỳ N: Systolic Array thực hiện kết nối và độc quyền đọc dữ liệu từ phân vùng PING để thực hiện các phép toán nhân cộng. Đồng thời, lõi điều khiển DMA ngoại vi thực hiện ghi dữ liệu của khối phân khối tiếp theo ($Tile_{N+1}$) từ bộ nhớ DDR RAM ngoài vào phân vùng PONG.

Tại chu kỳ N+1: Ngay sau khi mảng tính toán xong và DMA hoàn thành việc nạp dữ liệu, mạch điều khiển FSM thực hiện đảo chéo trạng thái bộ chọn. Lúc này, mảng tâm thu sẽ đọc dữ liệu mới từ phân vùng PONG, trong khi lõi DMA tiếp tục ghi gối đầu dữ liệu của khối kế tiếp vào phân vùng PING.

Giải pháp này cho phép che giấu hoàn toàn độ trễ truyền dữ liệu của hệ thống Bus, đảm bảo mảng tính toán luôn được cung cấp dữ liệu liên tục ở mức thông lượng tối đa [9], [14].

2.7. Lõi truyền dữ liệu AXI-Datamover.

Để truyền dữ liệu giữa bộ nhớ DDR thuộc PS và bộ tăng tốc GEMM trong PL, hệ thống sử dụng IP AXI DataMover của AMD thay cho AXI DMA. AXI DataMover là lõi truyền dữ liệu tốc độ cao hỗ trợ chuyển đổi giữa giao thức AXI4 Memory-Mapped (AXI4-MM) và AXI4-Stream, cho phép truyền trực tiếp dữ liệu giữa bộ nhớ và các khối xử lý dạng luồng mà không cần bộ xử lý ARM phải tham gia vào quá trình sao chép dữ liệu. IP hỗ trợ hai kênh truyền độc lập gồm Memory-Mapped to Stream (MM2S) và Stream to Memory-Mapped (S2MM), đồng thời tự động sinh địa chỉ truy cập bộ nhớ, thực hiện các giao dịch AXI burst và quản lý quá trình truyền dữ liệu theo các lệnh được gửi từ phần mềm thông qua giao diện điều khiển AXI4-Lite [15].

So với AXI DMA, AXI DataMover có cấu trúc đơn giản và linh hoạt hơn do chỉ cung cấp chức năng truyền dữ liệu mức thấp, không tích hợp bộ điều khiển Scatter-Gather hay các cơ chế quản lý descriptor trong phần cứng [16]. Điều này cho phép hệ thống chủ động xây dựng Matrix Tile Controller để tự điều phối quá trình phân khối ma trận, phát sinh địa chỉ DDR và gửi các lệnh truyền dữ liệu phù hợp với từng tile. Nhờ đó, thiết kế giảm được tài nguyên phần cứng sử dụng, tăng khả năng tùy biến luồng truyền dữ liệu và phù hợp với kiến trúc bộ tăng tốc GEMM được đề xuất, trong khi vẫn duy trì băng thông truyền dữ liệu cao giữa PS và PL.

2.8. Giao thức giao tiếp Bus họ AMBA AXI4

Trong kiến trúc hệ thống SoC hiện đại, họ giao tiếp AMBA AXI4 (Advanced eXtensible Interface 4) do ARM định nghĩa đóng vai trò là xương sống kết nối các thành phần IP phần cứng. Tùy thuộc vào đặc tính băng thông, yêu cầu tài nguyên logic và bản

chất của dữ liệu truyền tải, hệ thống tích hợp linh hoạt cả ba biến thể giao thức bus bao gồm: AXI4-Memory Mapped (AXI4-MM), AXI4-Lite và AXI4-Stream [17], [18].

2.8.1. Giao thức AXI4-Memory Mapped

AXI4-Memory Mapped (thường gọi là AXI4 Full) là giao thức bus định hướng bộ nhớ, được thiết kế cho các tác vụ yêu cầu ánh xạ địa chỉ tường minh, hiệu năng cao và băng thông lớn. Kiến trúc của AXI4-MM phân tách hoàn toàn quá trình truyền nhận dữ liệu thành 5 kênh độc lập, mỗi kênh sở hữu một tập hợp các tín hiệu bắt tay độc lập:

Kênh địa chỉ ghi (Write Address Channel)

Kênh dữ liệu ghi (Write Data Channel)

Kênh đáp ứng ghi (Write Response Channel)

Kênh địa chỉ đọc (Read Address Channel)

Kênh dữ liệu đọc (Read Data Channel)

Nhờ cấu trúc song song này, AXI4-MM hỗ trợ khả năng truyền nhận dữ liệu hai chiều toàn phần cùng lúc. Đặc tính cốt lõi của AXI4-MM là hỗ trợ cơ chế truyền hàng loạt: bên Master chỉ cần phát ra địa chỉ cơ sở một lần duy nhất trong chu kỳ thiết lập địa chỉ ban đầu, sau đó có thể truyền liên tiếp một chuỗi tối đa lên đến 256 Data Words mà không cần lặp lại pha địa chỉ. Giao thức này cũng hỗ trợ các tính năng nâng cao như Out-of-order transactions và xử lý Data interleaving nhằm khai thác tối đa hiệu suất bus [17].

2.8.2. Giao thức AXI4-Lite

AXI4-Lite là một tập hợp con được đơn giản hóa từ giao thức gốc AXI4-MM. Điểm khác biệt lớn nhất của AXI4-Lite là loại bỏ hoàn toàn khả năng truyền dữ liệu hàng loạt. Điều này đồng nghĩa với việc mỗi giao dịch đọc hoặc ghi chỉ cho phép dịch chuyển một từ dữ liệu duy nhất cho mỗi chu kỳ địa chỉ. Giao thức này vẫn duy trì cấu trúc 5 kênh độc lập nhưng lược bỏ phần lớn các tín hiệu điều khiển phụ phức tạp [16].

Do cấu trúc bus tối giản, AXI4-Lite chiếm dụng rất ít tài nguyên phần cứng và đơn giản hóa đáng kể thiết kế mạch logic điều khiển. Trong hệ thống, AXI4-Lite đóng vai trò lý tưởng để PS truy cập, cấu hình và giám sát các thanh ghi chức năng (Control and Status Registers - CSR) hoặc các thiết bị ngoại vi tốc độ thấp có ánh xạ không gian địa chỉ tĩnh trong PL [17], [18].

2.8.3. Giao thức AXI4-Stream

Khác biệt hoàn toàn với hai giao thức ánh xạ bộ nhớ nêu trên, giao thức AXI4-Stream được thiết kế chuyên dụng cho các luồng dữ liệu một chiều tốc độ cao và không định hướng không gian địa chỉ. Bằng việc loại bỏ hoàn toàn các kênh và tín hiệu quản

lý địa chỉ, AXI4-Stream tối giản hóa tối đa cấu trúc bus để tập trung toàn bộ băng thông vật lý cho việc truyền tải luồng dữ liệu nối tiếp giữa Master và Slave [19].

Cơ chế điều luồng của AXI4-Stream dựa trên một chu kỳ bắt tay duy nhất thông qua hai tín hiệu điều khiển chủ đạo tích cực ở mức cao:

TVALID: Do Master kéo lên mức cao để báo hiệu rằng dữ liệu hiện tại đang hiện diện trên đường bus đã hợp lệ.

TREADY: Do Slave kéo lên mức cao để báo hiệu rằng nó đã sẵn sàng tiếp nhận dữ liệu từ bus.

Giao dịch truyền dữ liệu thực tế chỉ xảy ra tại sườn lên của xung nhịp khi cả hai tín hiệu TVALID và TREADY đồng thời ở trạng thái logic bằng 1 [18]. Kết hợp với tín hiệu TLAST dùng để đánh dấu phần tử cuối cùng của một block hoặc một gói dữ liệu, bus AXI4-Stream cho phép đẩy liên tục một Data Word trên mỗi chu kỳ xung nhịp hệ thống mà không gặp phải độ trễ do chuyển đổi địa chỉ. Nhờ đó, hệ thống khai thác được tối đa băng thông truyền tải giữa lõi DMA và khối xử lý tăng tốc phần cứng trong mạch PL.

2.9. Tái chuẩn hóa và đánh giá

Sau khi mảng tính toán hoàn thành việc tích lũy dọc theo chiều sâu cấu trúc phân khối, các kết quả trung gian mang định dạng INT32 lưu trữ tại các hàng PE sẽ được đẩy tuần tự vào bộ nhớ đệm FWFT FIFO nhằm tối ưu hóa chu kỳ đọc không độ trễ. Tại đây, module xử lý hậu kỳ thực hiện phép tính toán dịch bit làm tròn kết hợp mạch bão hòa nhằm ép kiểu dữ liệu từ INT32 quay trở lại định dạng INT8 [4]. Quá trình này được thực hiện thông qua bộ Rescale Pipeline phần cứng mô phỏng chính xác thuật toán dịch bit cố định của mạch RTL:

$$\text{Bias Rounding} = 1 \ll (\text{SCALE_SHIFT} - 1)$$

$$\text{Data}_{\text{shifted}} = (\text{Data}_{\text{int32}} + \text{Bias Rounding}) \gg \text{SCALE_SHIFT}$$

$$\text{Data}_{\text{int8}} = \text{clamp}(\text{Data}_{\text{shifted}} - 128{,}127)$$

Hệ thống sau khi tích hợp toàn diện sẽ được tiến hành đánh giá hiệu năng dựa trên hai trục chỉ số chính:

1. Hiệu năng tốc độ xử lý phần cứng: Tiến hành đo đạc và so sánh trực tiếp thời gian thực thi phép nhân ma trận kích thước lớn khi chạy thuần túy bằng phần mềm trên CPU so với thời gian xử lý khi kích hoạt bộ tăng tốc trên FPGA nhằm chứng minh gia tốc thông lượng. Đồng thời cũng đo các giá trị thời gian như Transfer Time, Compute Time để đánh giá tốc độ của các thành phần trong hệ thống.

2. Kiểm chứng chức năng chính xác: Toàn bộ tệp nhị phân đầu ra thu được từ kit KV260 sẽ được nạp ngược lại môi trường Python phần mềm. Tại đây, một mô hình toán học kiểm chứng chéo (Golden Model) được thiết lập bằng thư viện NumPy/PyTorch để thực hiện phép toán GEMM tương đương trên các ma trận gốc. Sau đó, thực hiện 2 việc: giải lượng tử kết quả để so sánh sai số với kết quả gốc FP32 và đối chiếu sai lệch bit.

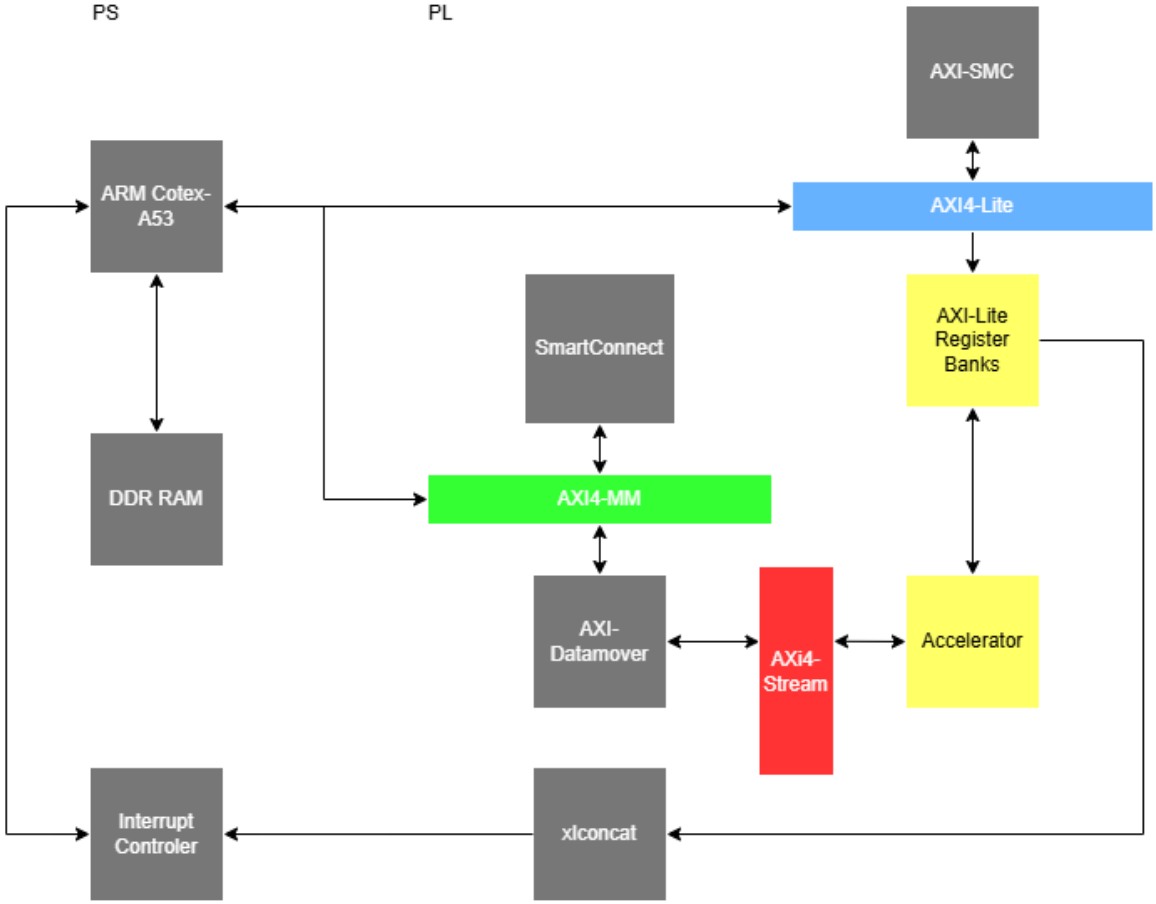
Các chỉ số đánh giá sai số từ kết quả giải lượng tử hóa là Cosine Similarity, SQNR, MAE và RMSE (tương tự như các chỉ số khi đánh giá sai số của dữ liệu lượng tử hóa) nhằm kiểm tra nhiễu lượng tử và giữ được các đặc trưng của mô hình.

Việc đối chiếu trực tiếp không sai lệch (Bit-True Cross Verification) giữa ma trận phần cứng và ma trận phần mềm đảm bảo tính đúng đắn tuyệt đối của thiết kế mạch logic phần cứng trước khi tích hợp vào hệ thống lớn.

Chương II. KIẾN TRÚC HỆ THỐNG

Đồ án thiết kế khai thác đồng thời PS và PL của KV260. Trong kiến trúc này, PS đảm nhiệm việc khởi tạo hệ thống, cấu hình các tham số tính toán, quản lý vùng nhớ DDR và điều khiển quá trình truyền dữ liệu. Toàn bộ phép nhân ma trận được thực hiện trên PL thông qua bộ tăng tốc GEMM được thiết kế bằng HDL nhằm khai thác khả năng xử lý song song của FPGA.

Để kết nối giữa PS và PL, hệ thống sử dụng kiến trúc AMBA AXI với ba giao thức chính. AXI4-Lite được dùng để cấu hình các thanh ghi điều khiển của bộ tăng tốc, bao gồm kích thước ma trận, tham số Rescale, địa chỉ vùng nhớ và tín hiệu khởi động hệ thống. AXI4 Memory-Mapped (AXI4-MM) đóng vai trò truyền dữ liệu giữa bộ nhớ DDR và AXI DataMover, trong khi AXI4-Stream được sử dụng bên trong PL để truyền liên tục các dòng dữ liệu giữa DataMover và bộ tăng tốc mà không cần thông tin địa chỉ, giúp tối đa hóa thông lượng truyền.



Hình 6. Tổng quan hệ thống

Các ma trận đầu vào được lưu trong bộ nhớ DDR của hệ thống PS. Trước khi bắt đầu tính toán, chương trình chạy trên ARM Cortex-A53 ghi các tham số cấu hình, địa chỉ vùng nhớ và tín hiệu điều khiển vào khối AXI4-Lite Register thông qua giao thức AXI4-Lite. Các thanh ghi này được kết nối trực tiếp với bộ tăng tốc để thiết lập chế độ hoạt động của hệ thống.

Sau khi nhận lệnh khởi động, AXI DataMover đọc dữ liệu từ DDR thông qua giao thức AXI4-MM, chuyển đổi thành luồng AXI4-Stream và truyền trực tiếp đến bộ tăng tốc gemm_top_l2. Sau khi hoàn thành phép nhân ma trận, kết quả được xuất trở lại dưới dạng AXI4-Stream, sau đó DataMover ghi ngược về DDR thông qua kênh S2MM.

Trong suốt quá trình xử lý, CPU không trực tiếp tham gia truyền dữ liệu mà chỉ thực hiện cấu hình và giám sát trạng thái của hệ thống. Khi một phiên truyền hoàn tất, các tín hiệu ngắt từ DataMover được tổng hợp bởi xlconcat trước khi đưa tới bộ điều khiển ngắt của PS. CPU nhận tín hiệu này để xác nhận quá trình xử lý đã hoàn thành hoặc tiếp tục cấu hình cho lần thực thi kế tiếp.

Chương III. THIẾT KẾ HỆ THỐNG PHẦN CỨNG.

1. Thiết kế bộ tăng tốc nhân ma trận.

Lõi của hệ thống là module gemm_top_l2, đóng vai trò bao gói toàn bộ các phân hệ xử lý bên trong vùng PL của chip Zynq UltraScale+. Dữ liệu chảy qua bộ tăng tốc theo kiến trúc đường ống nhiều tầng, được thiết kế chuyên biệt nhằm tối đa hóa hiệu năng tính toán và tận dụng triệt để các tài nguyên phần cứng đặc thù — bao gồm khối DSP48E2, BRAM, UltraRAM — của nền tảng FPGA.

Toàn bộ kiến trúc phần cứng được phân chia thành hai lớp trừu tượng. Lớp ngoài cùng do module l2_tiling_agu đảm nhận, chịu trách nhiệm quản lý luồng dữ liệu quy mô lớn từ bộ nhớ DDR thông qua AXI Datamover, thực hiện phân chia ma trận theo từng khối và điều phối song song giữa quá trình nạp dữ liệu và tính toán. Lớp bên trong do module gemm_compute_core thực thi phép nhân ma trận thực sự trên mảng tâm thu 16×16. Hai lớp này kết hợp thành một hệ thống hoàn chỉnh được bọc ngoài bởi gemm_top_l2 và giao tiếp với CPU thông qua khối thanh ghi AXI4-Lite tùy chỉnh gemm_l2_axi_lite_regs.

1.1. Module l2_tiling_agu – Điều phối luồng dữ liệu cấp khối

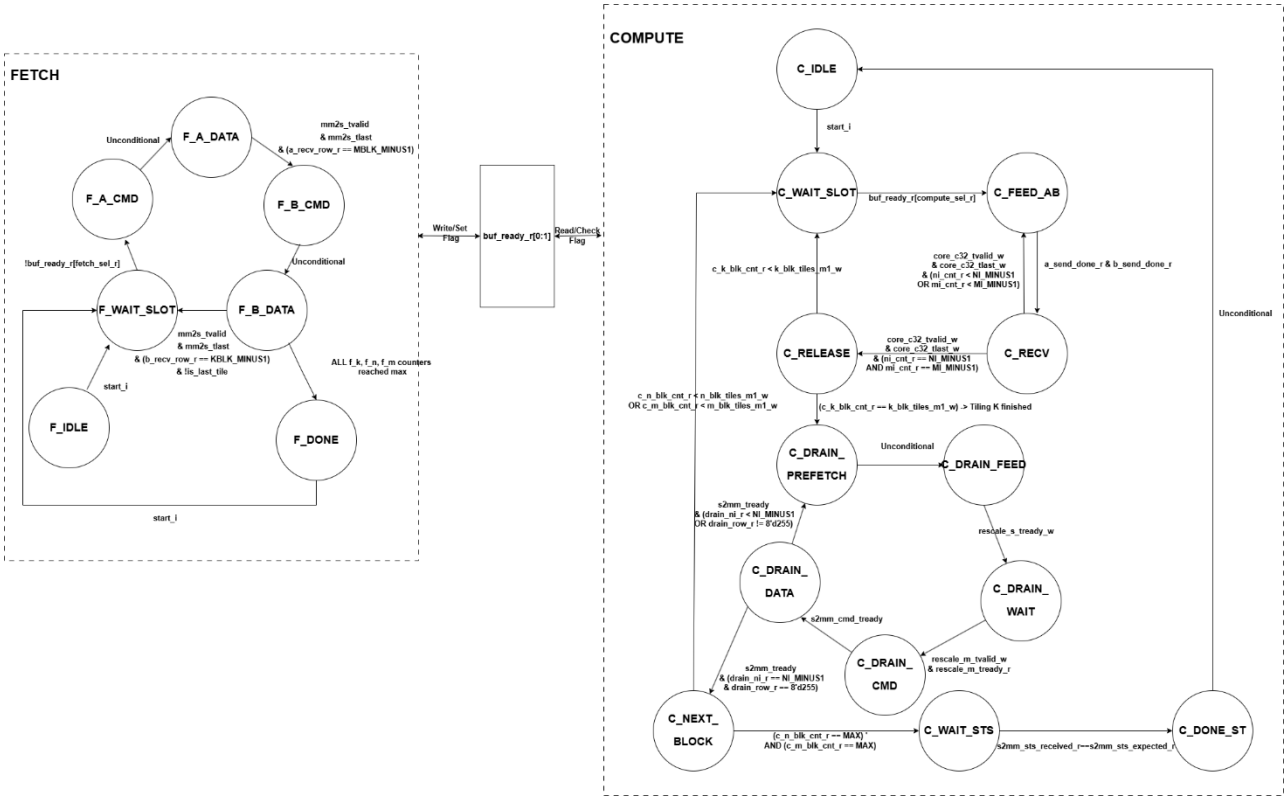
Phép nhân ma trận GEMM tổng quát với kích thước linh động không thể được nạp toàn bộ vào bộ nhớ on-chip của FPGA. Giải pháp bắt buộc là phân chia ma trận thành các khối nhỏ hơn, nạp từng khối từ DDR vào on-chip buffer rồi tính toán tuần tự. Đây là thách thức lớn nhất về kiến trúc: làm thế nào để băng thông DDR không trở thành nút cổ chai kìm hãm mảng tâm thu.

Module l2_tiling_agu sử dụng cơ chế double-buffer với hai khe nhớ độc lập, cho phép hai tiến trình hoạt động hoàn toàn song song:

- Trạng thái Fetch: Điều khiển AXI DMA đọc khối ma trận A và B tiếp theo từ DDR vào khe nhớ đang rảnh (GBUF A và GBUF B trên on-chip BRAM/URAM).

- Trạng thái Compute: Đồng thời lấy dữ liệu từ khe nhớ đã sẵn sàng để đưa vào lõi tính toán gemm_compute_core.

Khi tiến trình Compute hoàn tất một khối, nó giải phóng khe nhớ, cho phép Fetch tiếp tục ghi khối kế tiếp. Hai tiến trình trao đổi vai trò thông qua con trỏ fetch_sel và compute_sel. Nhờ đó, độ trễ truy cập DDR được che giấu hoàn toàn sau thời gian tính toán của mảng tâm thu.



Hình 7. FSM của module l2_tiling_agu

1.1.1. Bộ đệm on-chip: GBUFA, GBUFB và C_ACCUM

Để đáp ứng băng thông đọc tốc độ cao cho mảng tâm thu, ma trận A được lưu trữ trong GBUFA — một tập gồm 16 BRAM song song, mỗi BRAM phục vụ một hàng của mảng tâm thu. Kiến trúc này cho phép đọc đồng thời 16 byte (một hàng nguyên vẹn của ma trận A) trong một chu kỳ clock, loại bỏ hoàn toàn tắc nghẽn băng thông đọc dữ liệu activation. Ma trận B được lưu trong GBUFB là một BRAM đơn rộng 128-bit, đủ để cung cấp 16 byte trọng số mỗi chu kỳ cho cột đầu vào của mảng.

Kết quả tích lũy giữa các K-block được lưu vào C_ACCUM — một UltraRAM 512-bit, được chọn vì dung lượng yêu cầu vượt quá giới hạn BRAM. Module sử dụng cổng read_first để đọc giá trị hiện tại và cộng dồn kết quả mới vào trước khi ghi lại, thực hiện phép tích lũy K-block trực tiếp trong phần cứng.

1.1.2. Chuỗi trạng thái FSM Compute

Sau khi tích lũy đủ tất cả K-block, trạng thái Compute chuyển sang giai đoạn xuất dữ liệu. Nó đọc từng hàng từ C_ACCUM, đưa qua module int32_int8_rescale để chuyển đổi từ INT32 về INT8, rồi phát lệnh AXI DMA S2MM để ghi từng đoạn 16 byte về DDR. Một cơ chế quan trọng được bổ sung là đếm xác nhận S2MM: module không báo done ngay khi phát xong lệnh DMA mà phải chờ đủ số tín hiệu s2mm_sts_tvalid xác nhận từ DMA, đảm bảo toàn bộ dữ liệu đã được ghi hoàn toàn vào DDR trước khi kết thúc tác vụ.

Trạng thái	Tên	Chức năng
C_IDLE	Khởi tạo	Chờ tín hiệu start từ CPU. Khởi tạo con trỏ địa chỉ và bộ đếm khối.
C_WAIT_SLOT	Chờ buffer sẵn sàng	Đợi cờ buf_ready[compute_sel] được Fetch dựng lên, xác nhận khối dữ liệu mới đã sẵn sàng.
C_FEED_AB	Nạp dữ liệu vào lõi	Phát luồng AXI-Stream từ GBUFA/GBUFB vào gemm_compute_core. Chờ a_send_done && b_send_done.
C_RECV	Nhận kết quả INT32	Nhận luồng INT32 từ gemm_compute_core và lưu vào C_ACCUM (ghi đè nếu K-block đầu tiên, cộng dồn nếu không).
C_RELEASE	Giải phóng buffer	Xóa cờ buf_ready để Fetch có thể dùng lại khe nhớ. Chuyển sang DRAIN nếu đã đủ K-block, ngược lại quay về WAIT_SLOT.
C_DRAIN_PREFETCH	Prefetch C_ACCUM	Phát địa chỉ đọc C_ACCUM trước 1 chu kỳ, bù cho độ trễ đọc của UltraRAM.
C_DRAIN_FEED	Đưa vào Rescale	Chuyển hàng INT32 từ C_ACCUM sang int32_int8_rescale qua handshake AXI-Stream.
C_DRAIN_WAIT	Chờ kết quả INT8	Chờ rescale_m_tvalid, thu kết quả INT8 và chuẩn bị địa chỉ DMA đích.
C_DRAIN_CMD	Phát lệnh DMA	Gửi lệnh S2MM cho AXI DMA với địa chỉ và độ dài (N bytes = 16 bytes).
C_DRAIN_DATA	Ghi dữ liệu DMA	Phát dữ liệu INT8 lên kênh S2MM. Lặp lại cho tất cả hàng và cột của output block.
C_NEXT_BLOCK	Chuyển khối tiếp	Cập nhật con trỏ ma trận C theo thứ tự M→N→K. Kết thúc toàn bộ nếu đã xong.
C_WAIT_STS	Chờ xác nhận DMA	Đợi s2mm_sts_received == s2mm_sts_expected. Đảm bảo DDR đã ghi hoàn chỉnh.
C_DONE_ST	Hoàn tất	Dựng cờ done, hạ cờ busy và quay về C_IDLE.

Bảng 2. Phân tích FSM Compute của l2_tiling_agu

1.2. Module gemm_l2_axi_lite_regs — Giao tiếp điều khiển từ CPU

Bộ tăng tốc cần nhận nhiều tham số cấu hình trước mỗi lần thực thi GEMM: kích thước ma trận, địa chỉ DDR của A, B, C, các tham số lượng tử hóa, và tín hiệu khởi động.

Module thực hiện một slave AXI4-Lite đầy đủ, cung cấp 13 thanh ghi 32-bit ánh xạ vào không gian địa chỉ liên tục từ 0x00 đến 0x30. CPU ghi cấu hình một lần trước khi khởi động, sau đó chỉ cần ghi bit start tại thanh ghi điều khiển 0x2C để phát xung kích hoạt một chu kỳ. Thanh ghi trạng thái tại cùng địa chỉ cho phép CPU thăm dò cờ busy và cờ done (được giữ bền — sticky — cho đến khi CPU đọc, tránh mất xung done nếu polling không đúng thời điểm). Ngoài ra, module còn phát tín hiệu ngắt khi done tích cực, cho phép CPU dùng mô hình interrupt thay vì polling liên tục, giảm tải xử lý phần mềm.

1.3. Module gemm_compute_core — Lõi tính toán cấp tile

Module này đóng gói toàn bộ đường ống tính toán từ dữ liệu đầu vào AXI-Stream đến kết quả INT32 đầu ra, bao gồm 7 phân hệ con kết nối nối tiếp: BRAM ping-pong buffer, bộ điều khiển luồng systolic, mạng làm lệch, mảng tâm thu 16×16, bộ tuần tự hóa đầu ra, FIFO đệm, và bộ tích lũy sau khi tính toán.

1.4. Module bram_input_buffer — Đệm đầu vào ping-pong

Luồng AXI-Stream từ Datamover truyền dữ liệu với tốc độ có thể không đồng đều, trong khi mảng tâm thu yêu cầu dữ liệu được truy xuất tuần tự theo địa chỉ với độ trễ xác định. Nếu kết nối trực tiếp, bất kỳ độ trễ nào từ phía Datamover cũng sẽ đình trệ toàn bộ mảng tâm thu. Ngoài ra, cần đảm bảo không có xung đột đọc/ghi đồng thời vào cùng một vùng nhớ.

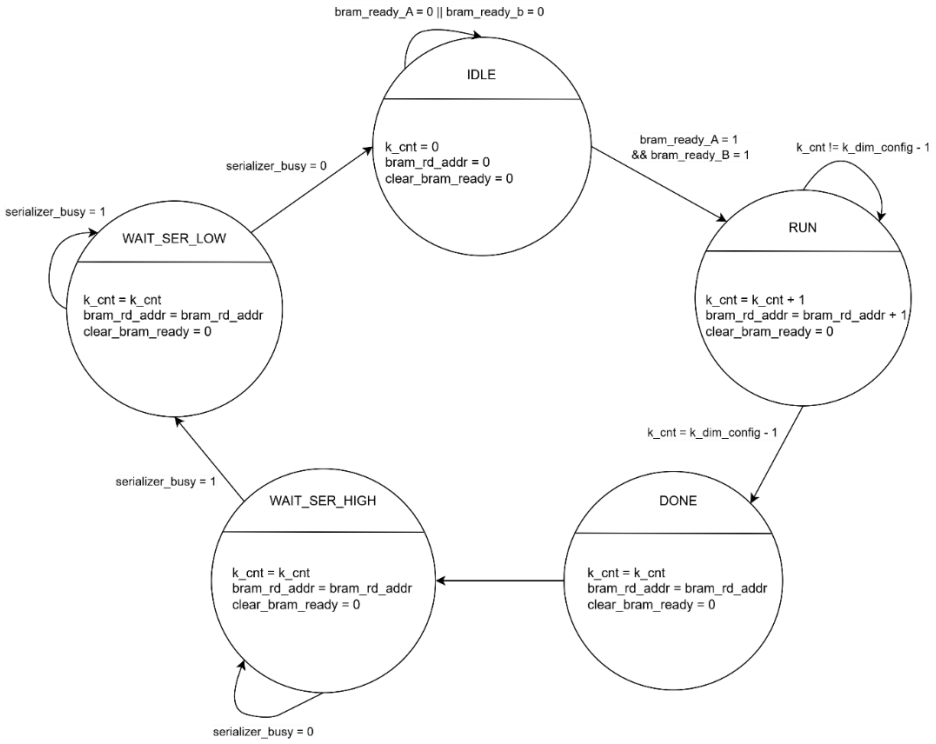
Module duy trì hai khối BRAM song song, được Vivado suy luận ra từ cấu trúc always block chuẩn nhờ thuộc tính (* ram_style = "block" *). Hai con trỏ wr_bank và rd_bank chỉ định mỗi thời điểm BRAM nào đang được Datamover ghi và BRAM nào đang được bộ điều khiển systolic đọc. Tín hiệu tready trên cổng AXI-Stream đầu vào được kết nối trực tiếp với bit đảo của cờ ready của bank đang ghi, tự động dừng Datamover khi bank đó đã đầy.

Khi bộ điều khiển systolic phát tín hiệu clear_ready báo đã đọc xong một tile, module đồng thời xóa cờ ready của bank đọc hiện tại và hoán đổi rd_bank. Cơ chế này đảm bảo mảng tâm thu luôn truy xuất được dữ liệu hợp lệ từ bank đã đầy, trong khi Datamover song song nạp dữ liệu tile tiếp theo vào bank còn lại. Đường đọc ra sử dụng thêm một thanh ghi pipeline ngoài thanh ghi đọc nội bộ của BRAM, tạo tổng cộng 2 chu kỳ trễ đọc nhất quán, giúp bộ điều khiển luồng tính toán đúng thời điểm dữ liệu xuất hiện tại đầu vào mảng.

1.5. Module systolic_dataflow_ctrl — Bộ điều khiển luồng mảng tâm thu

Mảng tâm thu không tự biết khi nào dữ liệu sẵn sàng, không tự tạo địa chỉ đọc BRAM, và không thể phối hợp với bộ tuần tự hóa đầu ra nếu không có điều phối trung tâm. Ngoài ra, các tín hiệu điều khiển phải đến đúng thời điểm dữ liệu tương ứng vào đến mảng, yêu cầu căn chỉnh trễ chính xác.

Module hiện thực một máy trạng thái hữu hạn 5 trạng thái điều phối toàn bộ chu kỳ tính toán. Các tín hiệu điều khiển curr_vld, curr_clr và curr_lst được tạo ra ngay tại thời điểm phát địa chỉ đọc, sau đó đi qua hai tầng thanh ghi delay để bù đúng 2 chu kỳ trễ của đường đọc BRAM. Kết quả là dữ liệu và tín hiệu điều khiển đến mảng tâm thu cùng một chu kỳ clock.



Hình 8. FSM của module systolic_dataflow_ctrl

Trạng thái	Tên trạng thái	Chức năng chi tiết
3'd0	IDLE	Đứng chờ cho đến khi cả 2 khối BRAM A và B đều sẵn sàng dữ liệu (bram_ready_A && bram_ready_B). Khi thỏa mãn, reset bộ đếm và chuyển sang RUN.
3'd1	RUN	Tăng địa chỉ đọc BRAM (bram_rd_addr) và bộ đếm k_cnt mỗi chu kỳ. Tạo curr_vld=1 liên tục, curr_clr=1 tại k_cnt==0, curr_lst=1 tại k_cnt==k_dim-1. Khi k_cnt đạt giới hạn, phát clear_bram_ready và chuyển sang DONE.
3'd2	DONE	Trạng thái đệm 1 chu kỳ, tự động chuyển ngay sang WAIT_SER_HIGH ở xung clock tiếp theo.

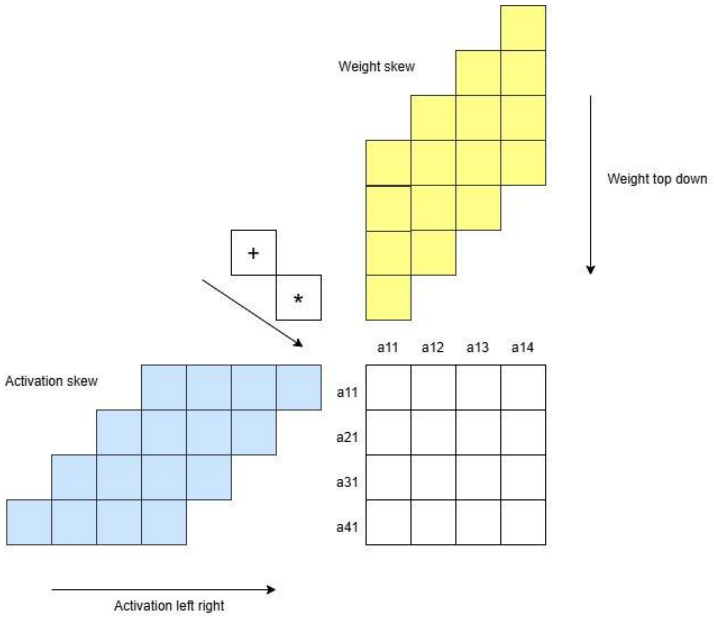
3'd3	WAIT_SER_HIGH	Đứng chờ cho đến khi output_serializer dựng cờ serializer_busy=1, xác nhận quá trình gom dữ liệu đã bắt đầu.
3'd4	WAIT_SER_LOW	Tiếp tục đứng chờ cho đến khi serializer_busy hạ về 0, toàn bộ kết quả của tile hiện tại đã được ghi vào FIFO. Sau đó quay về IDLE.

Bảng 3. Phân tích FSM systolic_dataflow_ctrl

Hai trạng thái chờ WAIT_SER_HIGH và WAIT_SER_LOW là đặc điểm thiết kế quan trọng: chúng ngăn bộ điều khiển khởi động tile tiếp theo trong khi bộ tuần tự hóa vẫn đang xả dữ liệu từ tile hiện tại vào FIFO, tránh xung đột trên đường dữ liệu đầu ra.

1.6. Module skew_network — Mạng làm lệch dữ liệu

Trong mảng tâm thu kiểu output-stationary, mỗi PE[i][j] phải nhận đúng phần tử A[i][k] và B[k][j] tại cùng một chu kỳ clock để phép nhân-cộng cho ra kết quả đúng. Tuy nhiên, dữ liệu từ BRAM được đọc ra đồng thời cho cả 16 hàng/cột. Nếu nạp song song, PE[0][0] sẽ nhận A[0][0] và B[0][0] đúng, nhưng PE[1][0] sẽ nhận A[1][0] kết hợp nhầm với B[0][0] thay vì phải chờ B[1][0] đến sau một chu kỳ.



Hình 9. Minh họa skew network cho ma trận 4x4

Module sinh ra bằng cấu trúc generate-for với tham số N=16 vòng lặp. Mỗi luồng dữ liệu thứ i bao gồm dữ liệu và tín hiệu điều khiển được đưa qua i tầng thanh ghi dịch. Kết quả là luồng dữ liệu hàng/cột thứ 0 đến ngay, hàng/cột thứ 1 trễ 1 chu kỳ, hàng/cột thứ 15 trễ 15 chu kỳ. Tập hợp 16 luồng bị dịch này tạo ra dạng sóng hình bậc thang theo thời gian, đảm bảo A[i][k] và B[k][j] gặp nhau chính xác tại PE[i][j] vào đúng chu kỳ

k. Việc trì hoãn cả tín hiệu điều khiển đi kèm với dữ liệu là điều kiện bắt buộc để cờ clear_acc và last_mac luôn đúng pha với phần tử dữ liệu tương ứng tại mỗi PE.

1.7. Lõi tính toán: mac_core và pe_wrapper

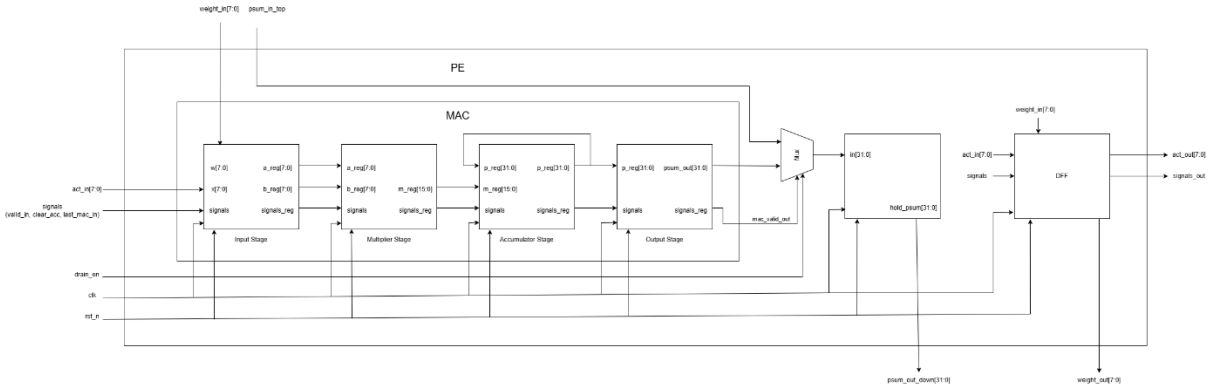
1.7.1. mac_core — Tối ưu cho DSP48E2

Module mac_core thực hiện phép nhân hai toán hạng INT8 có dấu và cộng dồn kết quả vào thanh ghi 32-bit. Thiết kế đặc biệt chú trọng vào việc tương thích cấu trúc đường ống 3 tầng (input register → multiplier register → accumulator register) với các tầng nội bộ A1/B1, MREG và PREG của khối cứng DSP48E2 trên Xilinx. Thuộc tính (* use_dsp = "yes" *) được gán trực tiếp để Vivado ưu tiên suy luận ra DSP48E2 thay vì dùng LUT.

Các thanh ghi datapath a_reg, b_reg, m_reg, p_reg không được reset, vì DSP48E2 không hỗ trợ reset đồng bộ hiệu quả trên các thanh ghi nội bộ. Chỉ các tín hiệu điều khiển v_reg, l_reg, c_reg được reset để đảm bảo hành vi xác định sau khởi động. Kết quả psum_out chỉ được cập nhật khi đồng thời v_reg3=1 và l_reg3=1, tức là chu kỳ dữ liệu hợp lệ cuối cùng đã đi qua đầy đủ 3 tầng pipeline. Cờ clear_acc khi được dựng tại đầu một tile mới sẽ thay thế p_reg bằng m_reg thay vì cộng dồn, khởi tạo lại bộ tích lũy mà không cần reset phần cứng.

1.7.2. pe_wrapper — Phần tử xử lý với luồng drain

Ngoài việc bao gói mac_core, pe_wrapper quản lý hai nhiệm vụ bổ sung. Thứ nhất, nó lưu trữ kết quả tích lũy trong thanh ghi hold_psum: khi mac_valid_out=1 - MAC xong một tile, kết quả được chốt vào hold_psum; khi drain_en=1 - đang xả kết quả, hold_psum nhận giá trị từ PE phía trên. Cơ chế này tạo thành chuỗi băng chuyền dọc: khi drain_en được kích hoạt, kết quả từ PE hàng 0 dịch xuống hàng 1, từ hàng 1 xuống hàng 2, và tiếp tục cho đến khi kết quả từ tất cả 16 hàng lần lượt xuất hiện tại đầu ra hàng dưới cùng. Thứ hai, pe_wrapper chuyển tiếp dữ liệu activation theo chiều ngang và trọng số theo chiều dọc, duy trì luồng dữ liệu systolic xuyên suốt toàn bộ mảng.



Hình 10. Thiết kế cụ thể cho module pe_wrapper

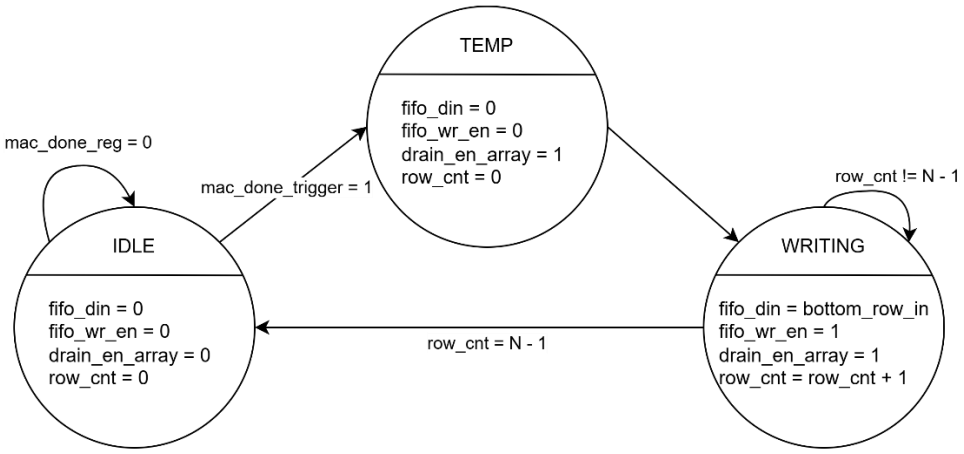
1.8. Module systolic_array_os — Mảng tâm thu 16×16

Module kết nối 256 phần tử pe_wrapper thành lưới 16×16 thông qua cấu trúc generate lồng nhau. Ma trận activation A đi theo chiều ngang từ trái sang phải, ma trận weight B đi theo chiều dọc từ trên xuống dưới; mỗi PE nhận một phần tử từ mỗi chiều, nhân chúng lại và tích lũy vào hold_psum nội bộ. Điều này loại bỏ hoàn toàn việc phải ghi kết quả trung gian ra bộ nhớ ngoài, giảm đáng kể tiêu thụ băng thông và năng lượng.

Tín hiệu drain_en_array được phát đồng thời đến tất cả 256 PE. Khi drain_en=1, chuỗi băng chuyền dọc trong từng cột bắt đầu hoạt động: sau N=16 chu kỳ, kết quả của tất cả 16 hàng trong một cột lần lượt xuất hiện tại bottom_row_out[j*32+:32]. Tín hiệu mac_done_trigger được kéo từ cờ valid_out của PE góc phải dưới cùng (hàng 15, cột 15) — PE hoàn thành tính toán muộn nhất do nhận dữ liệu trễ nhất sau skew network — làm tín hiệu kích hoạt quá trình xả toàn mảng.

1.9. Module output_serializer — Bộ tuần tự hóa đầu ra

Sau khi drain_en kích hoạt, mỗi chu kỳ clock đáy mảng xuất một vector 512-bit. Bộ FIFO phía sau chỉ nhận một vector mỗi lần, nhưng cần biết chính xác khi nào bắt đầu ghi và ghi bao nhiêu hàng. Một thách thức thực tế là tại chu kỳ đầu tiên khi drain_en được bật, dữ liệu tại bottom_row_out vẫn là giá trị cũ của hàng 15, chưa phải dữ liệu hợp lệ đầu tiên của tile mới.



Hình 11. FSM của module output_serializer

Trạng thái	Tên trạng thái	Chức năng chi tiết

1'd0	**IDLE**	Đứng chờ. Khi có tín hiệu kích hoạt (mac_done_trigger == 1), hệ thống chuẩn bị chuyển trạng thái. Các cờ dữ liệu vẫn ở mức 0.
	TEMP (hoặc SETUP)	Trạng thái đệm ẩn (1 chu kỳ). Bắt đầu bật cờ drain_en_array = 1 để kéo dữ liệu từ mảng tâm thu xuống. Tuy nhiên, ép cờ fifo_wr_en = 0 để tạo độ trễ 1 nhịp xung nhịp, ngăn FIFO ghi lặp dữ liệu cũ của hàng cuối cùng.
1'd1	**WRITING**	Bắt đầu chuỗi xả dữ liệu. Bật cờ fifo_wr_en = 1 để chính thức ghi vào FIFO. Tăng dần bộ đếm row_cnt. Khi xả đủ N hàng, FSM hạ các cờ điều khiển và quay lại IDLE.

Bảng 4. Phân tích FSM output_serializer

1.10. Module sc_fifo_fwft — Bộ đệm FIFO FWFT

Module FIFO được xây dựng trên nền tảng Xilinx XPM macro xpm_fifo_sync hoạt động ở chế độ FWFT. Trong chế độ này, phần tử đầu tiên được ghi vào FIFO tự động xuất hiện tại ngõ ra dout mà không cần thêm chu kỳ đọc phụ, cho phép post_accumulator kiểm tra dữ liệu và ra quyết định handshake ngay lập tức. Tín hiệu rd_en được điều kiện hóa bởi logic safe_fifo_rd = post_acc_s_tready & ~fifo_empty, ngăn việc đọc khi FIFO rỗng hoặc downstream chưa sẵn sàng.

1.11. Module post_accumulator — Tích lũy đa tile theo chiều K

Chiều K của phép GEMM thường lớn hơn nhiều so với chiều sâu K mà mảng tâm thu có thể xử lý trong một lần quét (k_dim ≤ BRAM_DEPTH). Cần có cơ chế tích lũy kết quả từ nhiều lần quét liên tiếp để hoàn thành đầy đủ phép nhân ma trận theo chiều K.

Module duy trì một vùng nhớ acc_ram gồm N=16 hàng, mỗi hàng chứa 16 phần tử INT32, được Vivado suy luận thành distributed RAM để đạt độ trễ đọc/ghi 1 chu kỳ. Một adder song song 16 chiều tính new_row_val mỗi chu kỳ: nếu tile_cnt==0, giá trị được ghi đè trực tiếp; nếu không, kết quả mới được cộng vào giá trị hiện tại trong acc_ram. Cơ chế này cho phép tích lũy chính xác kết quả từ num_k_tiles lần quét mà không cần xử lý phần mềm nào.

Trạng thái	Tên	Chức năng chi tiết
S_RECV (1'b0)	Nhận và tích lũy	Kéo dữ liệu từ FIFO (s_axis_tready=1). Mỗi hàng INT32 nhận được được ghi hoặc cộng dồn vào acc_ram[row_cnt]. Sau khi nhận đủ N hàng (row_cnt==N-1), kiểm tra tile_cnt: nếu chưa đủ

		num_k_tiles, tăng tile_cnt và tiếp tục; nếu đủ, chuyển sang S_OUTPUT.
S_OUTPUT (1'b1)	Xuất kết quả	Phát lần lượt N hàng từ acc_ram ra AXI-Stream downstream, theo thứ tự đảo ngược (acc_ram[N-1] trước, acc_ram[0] sau) để khôi phục thứ tự đúng vì drain theo chiều bottom-up. Handshake đúng chuẩn AXI: chỉ tăng out_cnt khi !tvalid || tready. Sau khi xuất xong, quay về S_RECV.

Bảng 5. Phân tích FSM post_accumulator

Tín hiệu m_axis_tuser được dựng ở hàng đầu tiên, m_axis_tlast được dựng ở hàng cuối cùng của mỗi lần xuất, giúp các tầng downstream nhận biết ranh giới khối ma trận kết quả.

1.12. Module int32_int8_rescale — Tái chuẩn hóa lượng tử

Sau quá trình tích lũy nhiều tile, mỗi phần tử kết quả là một số nguyên có dấu 32-bit. Truyền trực tiếp INT32 về DDR sẽ tiêu tốn băng thông gấp 4 lần so với INT8, đồng thời làm tăng dung lượng bộ nhớ và giảm hiệu quả xử lý của toàn hệ thống. Cần chuyển đổi về miền INT8 trên phần cứng mà không dùng bộ chia, vốn tiêu tốn nhiều LUT và có độ trễ lớn.

Module hiện thực toàn bộ chuỗi biến đổi theo dạng mạch tổ hợp song song cho tất cả N=16 phần tử đồng thời, được đăng ký hóa bởi một tầng thanh ghi đầu ra duy nhất. Chuỗi biến đổi cho mỗi phần tử gồm 5 bước:

1. Mở rộng dấu từ 32 lên 34 bit để tránh tràn số khi cộng bias.

2. Cộng round_bias = 1 << (scale_shift - 1) để làm tròn về giá trị gần nhất thay vì luôn làm tròn xuống.

3. Dịch phải số học scale_shift bit. Phép dịch phải số học bảo toàn bit dấu, tương đương phép chia cho 2^scale_shift với kết quả là số nguyên có dấu.

4. Cộng zero_point (mở rộng dấu từ 8 lên 34 bit) để dịch chuyển vùng biểu diễn theo lượng tử hóa không đối xứng nếu cần.

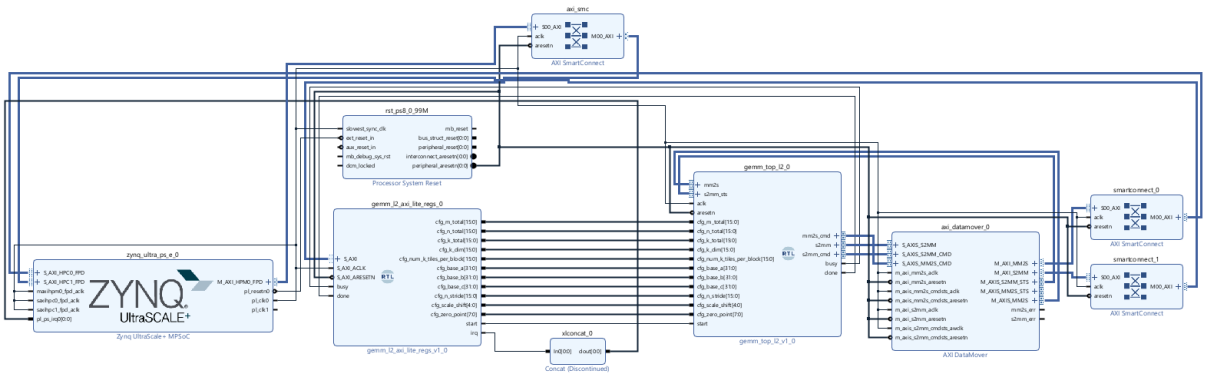
5. Xén biên bão hòa về miền [-128, 127] để đảm bảo kết quả luôn nằm trong phạm vi biểu diễn INT8 hợp lệ.

Giá trị scale_shift=10 được lựa chọn qua thực nghiệm với tập dữ liệu của hệ thống, tương đương phép chia cho 1024. Giá trị này tạo ra sự cân bằng hợp lý: giữ được độ phân giải số học của kết quả INT32 sau tích lũy mà không gây bão hòa quá nhiều phần tử. Toàn bộ chuỗi 5 bước này được hiện thực bằng phép dịch bit và cộng trừ đơn giản, tránh hoàn toàn việc dùng bộ chia phần cứng.

Module hỗ trợ back-pressure hoàn chỉnh: s_axis_tready chỉ được dựng khi ngõ ra rỗng hoặc downstream đang nhận (m_axis_tready=1). Khi bị stall, toàn bộ thanh ghi ngõ ra được giữ nguyên đến khi downstream sẵn sàng.

2. Triển khai hệ thống.

Hệ thống tăng tốc được triển khai trên nền tảng AMD Kria KV260 với bộ xử lý nhúng Zynq UltraScale+ MPSoC tích hợp CPU ARM Cortex-A53 và bộ tăng tốc phần cứng gemm_top_l2 trong vùng PL. Toàn bộ hệ thống hoạt động đồng bộ tại tần số 200 MHz.



Hình 12. Hệ thống chi tiết được triển khai trong Vivado

2.1. Kiến trúc giao tiếp

CPU và bộ tăng tốc trao đổi dữ liệu qua hai kênh phân biệt rõ ràng theo tốc độ và mục đích. Kênh cấu hình sử dụng giao thức AXI4-Lite, cho phép CPU ghi 11 tham số cấu hình (kích thước ma trận, địa chỉ DDR, tham số lượng tử hóa) vào khối thanh ghi gemm_l2_axi_lite_regs, sau đó phát lệnh start. Khối này còn cung cấp tín hiệu ngắt để CPU không cần polling liên tục, chuyển sang mô hình interrupt-driven tiết kiệm tài nguyên xử lý hơn.

Kênh dữ liệu tốc độ cao dùng giao thức AXI4 Memory-Mapped kết hợp AXI4-Stream thông qua AXI DMA hoạt động ở chế độ Scatter-Gather. DMA nhận lệnh 72-bit (địa chỉ + độ dài) từ l2_tiling_agu qua kênh MM2S Command và S2MM Command, tự truy cập DDR để đọc/ghi mà không cần CPU can thiệp. Các khối AXI SmartConnect thực hiện định tuyến, cân bằng tải và đồng bộ trên toàn bộ fabric bus, cho phép DMA và PS truy cập DDR song song mà không có xung đột.

2.2. Luồng thực thi tổng thể

Một phép GEMM hoàn chỉnh diễn ra theo trình tự sau: CPU ghi cấu hình và phát start; l2_tiling_agu khởi động tiến trình fetch đầu tiên, phát lệnh MM2S để đọc khối A và B từ DDR vào GBUFA và GBUFB; khi buffer sẵn sàng, tiến trình compute bắt đầu nạp dữ liệu qua gemm_compute_core; lõi tính toán xử lý từng tile K, tích lũy kết quả vào post_accumulator; sau khi đủ num_k_tiles, post_accumulator xuất hàng INT32 sang int32_int8_rescale và kết quả INT8 được ghi về DDR qua kênh S2MM; tiến trình fetch và compute chạy song song cho các khối tiếp theo. Khi toàn bộ ma trận C được ghi xong và mọi xác nhận S2MM đã nhận đủ, l2_tiling_agu dựng cờ done và ngắt được gửi về CPU.

3. Kiểm tra và phân tích.

3.1. Kiểm tra chức năng

Để kiểm tra tính chính xác của bộ tăng tốc, hệ thống được mô phỏng bằng testbench Register-Transfer Level (RTL) trên Vivado Simulator theo phương pháp end-to-end verification. Testbench thực hiện mô phỏng đầy đủ luồng dữ liệu từ DMA đầu vào, bộ tăng tốc General Matrix Multiply (GEMM) cho đến DMA đầu ra tương tự như khi hệ thống hoạt động thực tế trên FPGA.

Dữ liệu đầu vào được sinh ngẫu nhiên dưới định dạng số nguyên có dấu INT8 trong miền giá trị [-128, 127]. Hai ma trận: $A \in \mathbb{Z}^{16\times512}$ và $B \in \mathbb{Z}^{512\times16}$ được chia thành nhiều tile theo chiều K để truyền tuần tự vào bộ tăng tốc thông qua giao thức AXI4-Stream.

Đối với mỗi phần tử đầu ra, testbench đồng thời xây dựng một mô hình tham chiếu bằng phần mềm (golden model) để thực hiện: phép nhân cộng INT32, cộng dồn nhiều k-tile, tái chuẩn hóa về INT8, xén biên theo đúng thuật toán phần cứng.

Kết quả từ phần cứng sau khi xuất qua AXI4-Stream được lưu lại và so sánh trực tiếp với golden model. Hệ thống chỉ được xem là hợp lệ khi toàn bộ phần tử đầu ra khớp hoàn toàn với kết quả tham chiếu.

Kết quả mô phỏng console được thể hiện như sau:

```
================================================================
[150000 ns]  GEMM ACCELERATOR TESTBENCH
  N=16 | K_DIM=32 | K_BLOCKS=2 | NUM_K_TILES=16 | K_TOTAL=512
  SCALE_SHIFT=10 | ZERO_POINT=0
================================================================
[150000 ns] [SW]  Tinh golden C = A x B...
[150000 ns] [SW]  Golden model hoan thanh.
[150000 ns] [RX] Waiting for output matrix C...
[150000 ns] [TX]  Bat dau gui du lieu (16 tiles)...
[150000 ns] [TX]  Tile 1 / 16
[490000 ns] [TX]  Tile 2 / 16
[830000 ns] [TX]  Tile 3 / 16
[1170000 ns] [TX]  Tile 4 / 16
[2020000 ns] [TX]  Tile 5 / 16
[2880000 ns] [TX]  Tile 6 / 16
[3740000 ns] [TX]  Tile 7 / 16
[4600000 ns] [TX]  Tile 8 / 16
[5460000 ns] [TX]  Tile 9 / 16
[6320000 ns] [TX]  Tile 10 / 16
[7180000 ns] [TX]  Tile 11 / 16
[8040000 ns] [TX]  Tile 12 / 16
[8900000 ns] [TX]  Tile 13 / 16
[9760000 ns] [TX]  Tile 14 / 16
```

```
[10620000 ns] [TX]  Tile 15 / 16
[11480000 ns] [TX]  Tile 16 / 16
[12340000 ns] [TX]  Da nap du du lieu. Cho ket qua...
[14296000 ns] [RX] SOF nhan duoc - bat dau ghi matrix C...
[14446000 ns] [RX] Nhan du 16 hang.

================================================================
  GOLDEN MODEL COMPARISON
================================================================

----------------------------------------------------------------
  C_npu_result (INT8) - Hardware Output:
----------------------------------------------------------------
  Row[ 0]:   64   68   63 -128 -118  100 -115   12  127  126  127  127  117   46 -128  -71
  Row[ 1]:   51  -72  118   -1  127  -26  -43 -128   79   51  -99  -44  127  107    2  -22
  Row[ 2]:    2   31  127  -18 -128    3  -74   19 -128  127   61   59 -128  -40 -109  -90
  Row[ 3]:  -50    3   77  127   17   14   14 -128   -8   58  -45 -128   71   45  -28   92
  Row[ 4]:  -83  127  -77   58   35   28  -75  -52  -41  -46 -128 -128   -2  115   62  -50
  Row[ 5]:  109   76 -128 -128  127  127  -70  108   55  103  -13  -62    8   53 -103  127
  Row[ 6]: -128 -128  -22 -128  -27  127   95  127  -65  -50  -41  127  -66  -18  127  -90
  Row[ 7]:  100  -22   27  -71  -61   53  -66   22  127 -109  127   88    2   30 -126  -69
  Row[ 8]:   62    9 -124   25   47  -19  -35  -76 -128    8 -128 -128  127   54    5   80
  Row[ 9]: -108   74 -117  -81  107  -68  126 -128  127   81    3  -92   10  -81 -119  127
  Row[10]:  127  127  -71   34  -17   -5  104   -7  -91  127   39  127  -34  127 -128 -108
  Row[11]:   40 -128   26  127   19   98   -6 -128  -92   64  -86  127 -128  -20  127  108
  Row[12]:  111  -59  127 -103  -50 -128  106   59  127 -128   32 -128  -76 -128  -19   98
  Row[13]:  127  -47  127  -11   83  -11  -76 -128   84  127  -34  127   51 -128  -94  107
  Row[14]: -128   71  127  127   87    7   16  -12  127 -128 -128   -8  101 -128  100   85
  Row[15]:  -95   61  127  -20   41  106  121  127   73   39   67  127  -43  -10   47 -128

----------------------------------------------------------------
  C_expected (INT8) - Software Golden:
----------------------------------------------------------------
  Row[ 0]:   64   68   63 -128 -118  100 -115   12  127  126  127  127  117   46 -128  -71
  Row[ 1]:   51  -72  118   -1  127  -26  -43 -128   79   51  -99  -44  127  107    2  -22
  Row[ 2]:    2   31  127  -18 -128    3  -74   19 -128  127   61   59 -128  -40 -109  -90
  Row[ 3]:  -50    3   77  127   17   14   14 -128   -8   58  -45 -128   71   45  -28   92
  Row[ 4]:  -83  127  -77   58   35   28  -75  -52  -41  -46 -128 -128   -2  115   62  -50
  Row[ 5]:  109   76 -128 -128  127  127  -70  108   55  103  -13  -62    8   53 -103  127
  Row[ 6]: -128 -128  -22 -128  -27  127   95  127  -65  -50  -41  127  -66  -18  127  -90
  Row[ 7]:  100  -22   27  -71  -61   53  -66   22  127 -109  127   88    2   30 -126  -69
  Row[ 8]:   62    9 -124   25   47  -19  -35  -76 -128    8 -128 -128  127   54    5   80
  Row[ 9]: -108   74 -117  -81  107  -68  126 -128  127   81    3  -92   10  -81 -119  127
  Row[10]:  127  127  -71   34  -17   -5  104   -7  -91  127   39  127  -34  127 -128 -108
  Row[11]:   40 -128   26  127   19   98   -6 -128  -92   64  -86  127 -128  -20  127  108
  Row[12]:  111  -59  127 -103  -50 -128  106   59  127 -128   32 -128  -76 -128  -19   98
  Row[13]:  127  -47  127  -11   83  -11  -76 -128   84  127  -34  127   51 -128  -94  107
  Row[14]: -128   71  127  127   87    7   16  -12  127 -128 -128   -8  101 -128  100   85
  Row[15]:  -95   61  127  -20   41  106  121  127   73   39   67  127  -43  -10   47 -128
```

```
================================================================
  TEST SUMMARY
================================================================
  K_DIM=32 | NUM_K_TILES=16 | K_TOTAL=512
  SCALE_SHIFT=10 | ZERO_POINT=0
  SOF (tuser) OK : YES
  EOF (tlast) OK : YES
----------------------------------------------------------------
  [PASS]  TAT CA DUNG: 256 / 256 pixels!
================================================================
```

Bảng 6. Kết quả hiển thị trên Console

Kết quả này cho thấy:

Toàn bộ 256 phần tử của ma trận đầu ra *16 x 16* đều chính xác,

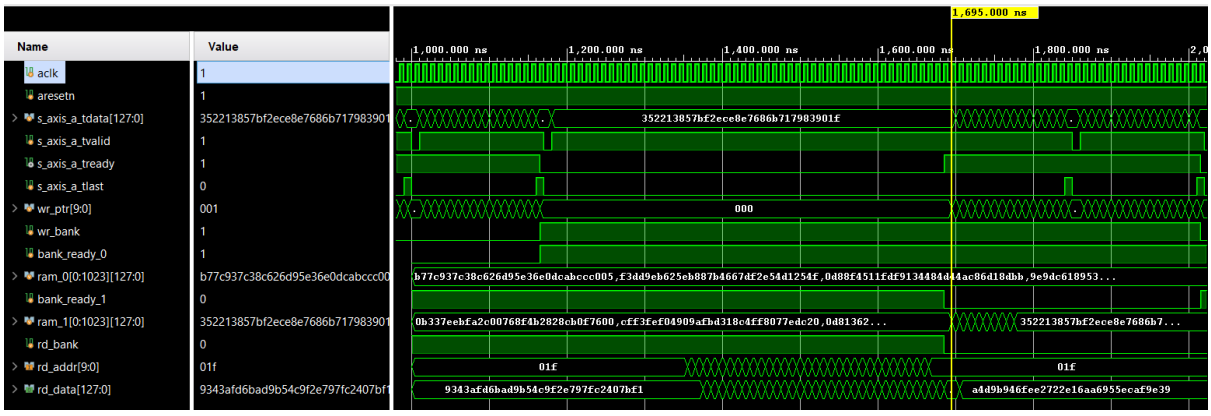
Kiến trúc Systolic Array hoạt động đúng,

Cơ chế cộng dồn nhiều K-tile chính xác,

Thuật toán requantization INT32 → INT8 hoạt động đúng với tham số scale_shift = 10,

Giao thức AXI4-Stream giữa các khối hoạt động ổn định.

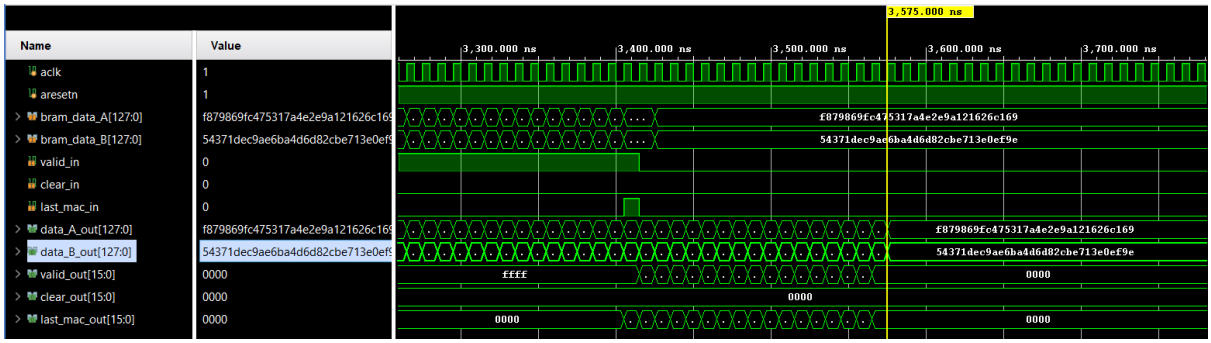
Trong quá trình mô phỏng, testbench cũng ghi nhận thời điểm bộ tăng tốc bắt đầu xuất dữ liệu kết quả thông qua tín hiệu tuser, cho phép xác nhận hoạt động đồng bộ giữa pipeline tính toán và khối DMA đầu ra.



Hình 13. Waveform ghi dữ liệu từ AXI DMA vào BRAM buffer

Hình 13 thể hiện quá trình nhận dữ liệu từ AXI DMA vào bộ đệm BRAM của ma trận A. Khi tín hiệu tvalid và tready đồng thời ở mức '1', dữ liệu trên bus tdata được chấp nhận và ghi vào BRAM sau một chu kỳ clock theo cơ chế synchronous write của FPGA. Trong quá trình này, BRAM vẫn đồng thời thực hiện đọc dữ liệu từ bank còn lại nhằm phục vụ tính toán song song của systolic array.

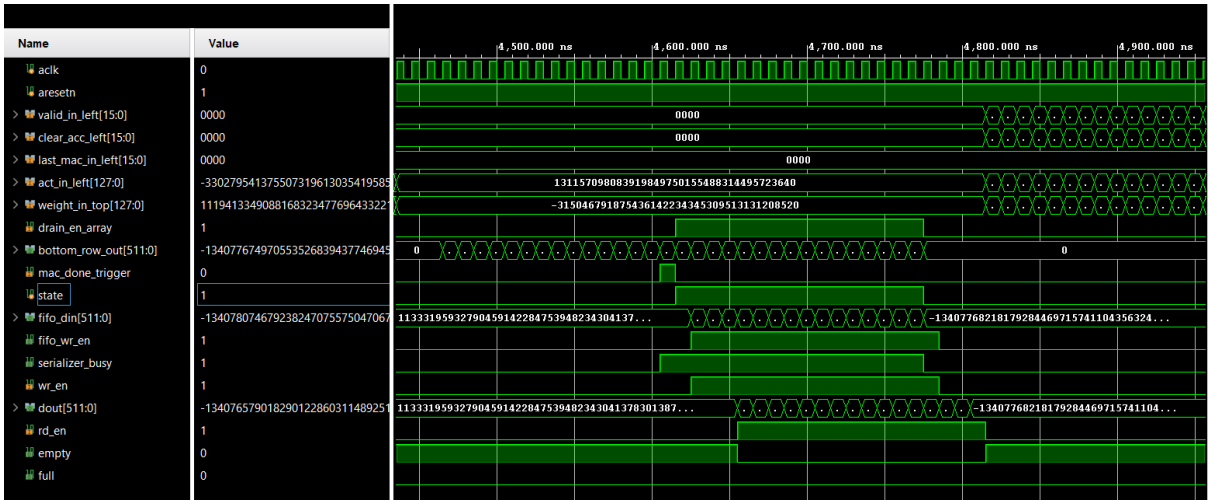
Ngoài ra, khi cả hai bank của ping-pong buffer đều ở trạng thái đầy (bank_ready = 1), tín hiệu tready sẽ tự động chuyển xuống mức ‘0’ để tạm dừng quá trình truyền dữ liệu từ DMA. Kết quả waveform cho thấy cơ chế AXI-Stream handshake và ping-pong buffer hoạt động chính xác theo thiết kế.



Hình 14. Waveform skew dữ liệu nhận từ BRAM

Hình 14 thể hiện waveform của khối systolic_dataflow_ctrl và skew_network trong quá trình đồng bộ dữ liệu đầu vào cho systolic array. Có thể thấy dữ liệu data_A_out và data_B_out được tạo độ trễ tuần tự trước khi đưa vào mảng PE, với độ trễ tối đa 16 chu kỳ clock tương ứng với số phần tử trong một hàng/cột của ma trận 16×16.

Ngoài dữ liệu, các tín hiệu điều khiển như valid_out, clear_out và last_mac_out cũng được dịch tuần tự thông qua các thanh ghi dịch nhằm đảm bảo dữ liệu và tín hiệu điều khiển luôn đồng bộ theo từng chu kỳ xử lý của systolic array. Kết quả waveform cho thấy cơ chế skew dữ liệu và đồng bộ pipeline hoạt động chính xác theo thiết kế.

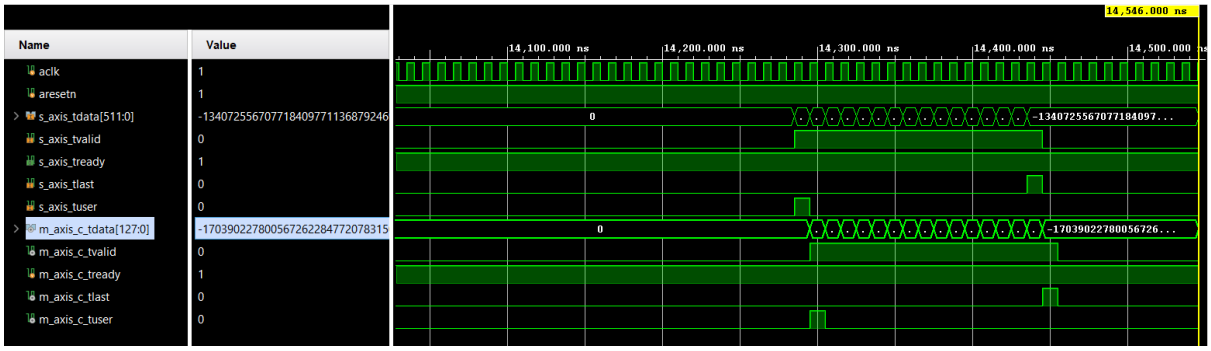


Hình 15. Waveform tín hiệu điều khiển SA và lưu kết quả vào FIFO

Hình 15 thể hiện waveform điều khiển quá trình xuất dữ liệu từ systolic array và lưu kết quả vào FIFO thông qua khối output_serializer. Sau khi quá trình MAC hoàn tất, tín hiệu mac_done_trigger được kích hoạt đồng thời với trạng thái serializer_busy

= 1, cho phép tín hiệu drain_en_array được bật để bắt đầu quá trình drain dữ liệu khỏi systolic array.

Đồng thời, FSM bên trong output_serializer chuyển trạng thái từ IDLE sang WRITING để ghi dữ liệu kết quả vào FIFO. Khoảng thời gian giữa tín hiệu wr_en = 1 và rd_en = 1 cho thấy FIFO hoạt động đúng theo cơ chế FWFT (First-Word Fall-Through), trong đó dữ liệu đầu tiên sẽ tự động xuất hiện ở ngõ ra mà không cần thêm chu kỳ đọc bổ sung. Kết quả waveform xác nhận quá trình điều khiển xuất dữ liệu và lưu trữ FIFO hoạt động chính xác theo thiết kế.



Hình 16. Waveform tái chuẩn hóa dữ liệu và truyền AXI-Stream output

Hình 16 thể hiện waveform của khối Rescale và quá trình truyền dữ liệu ra giao diện AXI-Stream output. Có thể thấy dữ liệu đầu ra và các tín hiệu điều khiển bị trễ một chu kỳ clock do khối Rescale cần thực hiện quá trình tái chuẩn hóa dữ liệu từ INT32 về INT8 trước khi xuất ra ngoài.

Sau khi quá trình rescale hoàn tất, dữ liệu sẽ được truyền thông qua giao diện AXI-Stream khi tín hiệu tvalid và tready đồng thời ở mức '1'. Kết quả waveform cho thấy quá trình tái chuẩn hóa và truyền dữ liệu output hoạt động chính xác theo thiết kế.

3.2. Phân tích PPA của hệ thống

Sau khi hoàn tất kiểm tra chức năng, hệ thống được đánh giá toàn diện theo ba tiêu chí PPA (Performance – Power – Area) nhằm xác định khả năng đáp ứng hiệu năng, mức sử dụng tài nguyên phần cứng và hiệu quả năng lượng của bộ tăng tốc GEMM trên nền tảng AMD Kria KV260.

3.2.1. Hiệu năng (Performance)

Đặc tính timing của thiết kế được đánh giá dựa trên kết quả phân tích sau quá trình Place & Route của Vivado. Hệ thống sử dụng một miền clock duy nhất clk_pl_0 với chu kỳ ràng buộc 5 ns, tương đương tần số 200 MHz. Kết quả timing report được tổng hợp trong bảng dưới đây.

Chỉ số	Giá trị	Ý nghĩa

WNS (Worst Negative Slack)	+0.017 ns	Đường tín hiệu setup chậm nhất vẫn còn dư 0.017 ns so với ràng buộc. Không có vi phạm setup.
TNS (Total Negative Slack)	0 ns	Tổng vi phạm setup trên toàn thiết kế bằng 0. Toàn bộ đường tín hiệu đáp ứng yêu cầu setup.
WHS (Worst Hold Slack)	+0.01 ns	Đường tín hiệu hold chặt nhất vẫn còn dư 0.01 ns. Không có vi phạm hold.
THS (Total Hold Slack)	0 ns	Tổng vi phạm hold trên toàn thiết kế bằng 0. Toàn bộ đường tín hiệu đáp ứng yêu cầu hold.

Bảng 7. Kết quả phân tích timing sau Implementation

Kết quả cho thấy thiết kế đạt timing closure hoàn toàn: WNS và WHS đều dương, TNS và THS đều bằng 0, nghĩa là không tồn tại bất kỳ vi phạm setup hay hold nào trên toàn bộ netlist. Tần số hoạt động 200 MHz cho thấy critical path còn dư biên an toàn. Điều này được giải thích bởi hai yếu tố kiến trúc chính: thứ nhất, các phép nhân INT8 được ánh xạ trực tiếp lên khối cứng DSP48E2 với cấu trúc pipeline 3 tầng sẵn có (input register → multiplier register → accumulator register), loại bỏ hoàn toàn đường nhân qua LUT vốn có độ trễ lớn; thứ hai, toàn bộ đường dữ liệu được phân đoạn bằng các thanh ghi giữa mỗi tầng xử lý, giữ cho mỗi đoạn tổ hợp đủ ngắn để đáp ứng chu kỳ clock nghiêm ngặt.

3.2.2. Tài nguyên phần cứng (Area)

Sau quá trình Implementation, Vivado tổng hợp báo cáo sử dụng tài nguyên trên chip Zynq UltraScale+ của KV260. Bảng dưới đây tóm tắt mức sử dụng của các loại tài nguyên chính.

Tài nguyên	Sử dụng	Khả dụng	Mức sử dụng (%)
LUT (Logic)	14554	117 120	12.43
LUTRAM	1 229	57 600	2.13
FF	29165	234 240	12.45
BRAM Tile	95.5	144	66.32
URAM	8	64	12.50
DSP	256	1 248	20.51

Bảng 8. Mức sử dụng tài nguyên FPGA sau Implementation

Ngoài thống kê tổng quát, Vivado cũng cho phép phân tích chi tiết phân bố tài nguyên theo từng module trong hệ thống. Kết quả được thể hiện trong Hình 17 bên dưới.

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	F8 Muxes (29280)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	URAM (64)	DSPs (1248)	BUFG_PS (96)	PS8 (1)
gemm_system_wrapper	14554	29165	445	264	128	4262	13325	1229	95.5	8	256	1	1
gemm_system_i (gemm_system)	14554	29165	445	264	128	4262	13325	1229	95.5	8	256	1	1
axi_datamover_0 (gemm_system_axi	1574	2058	18	0	0	320	1418	156	4.5	0	0	0	0
axi_smc (gemm_system_axi_smc_2)	749	800	5	7	0	156	748	1	0	0	0	0	0
gemm_l2_axi_lite_regs_0 (gemm_sys	93	292	0	0	0	88	93	0	0	0	0	0	0
gemm_top_l2_0 (gemm_system_gem	11145	24753	420	257	128	3600	10346	799	91	8	256	0	0
inst (gemm_system_gemm_top_l2	11145	24753	420	257	128	3600	10346	799	91	8	256	0	0
u_l2_tiling_agu (gemm_system	11145	24753	420	257	128	3600	10346	799	91	8	256	0	0
u_gemm_compute_core (g	6760	22809	86	257	128	3009	5961	799	30.5	0	256	0	0
u_bram_input_buffer_a (	601	142	2	256	128	128	601	0	8	0	0	0	0
u_bram_input_buffer_b (	104	142	4	0	0	29	104	0	8	0	0	0	0
u_output_serializer (gem	7	521	0	0	0	151	7	0	0	0	0	0	0
u_post_accumulator (ge	1118	572	8	1	0	173	604	514	0	0	0	0	0
u_sc_fifo_fwft_c (gemm_	56	51	64	0	0	84	56	0	14.5	0	0	0	0
u_skew_network (gemm_	299	0	0	0	0	54	14	285	0	0	0	0	0
u_systolic_array_os (gem	4528	21328	0	0	0	2652	4528	0	0	0	256	0	0
u_systolic_dataflow_ctrl	61	53	8	0	0	22	61	0	0	0	0	0	0
u_int32_int8_rescale (gemm	2966	129	224	0	0	490	2966	0	0	0	0	0	0
rst_ps8_0_99M (gemm_system_rst_ps	14	114	0	0	0	77	13	1	0	0	0	0	0
smartconnect_0 (gemm_system_sma	318	419	0	0	0	62	185	133	0	0	0	0	0
smartconnect_1 (gemm_system_sma	662	729	2	0	0	151	523	139	0	0	0	0	0

Hình 17. Phân bố tài nguyên theo từng IP/module trong hệ thống

Phân tích chi tiết theo module cho thấy phần lớn tài nguyên tập trung trong khối gemm_top_l2_0, trực tiếp phản ánh mức độ phức tạp của lõi tính toán.

Systolic Array (systolic_array_inst) là module tiêu thụ tài nguyên đặc trưng nhất với toàn bộ 256 khối DSP48E2 của hệ thống và 21 328 thanh ghi FF. Điều này hoàn toàn nhất quán với kiến trúc: mỗi trong số 256 phần tử PE được ánh xạ trực tiếp lên một DSP48E2, và lượng FF lớn phục vụ cho chuỗi thanh ghi pipeline 3 tầng bên trong mỗi mac_core cũng như các thanh ghi chuyển tiếp dữ liệu act_out, weight_out dọc theo hàng và cột.

Module u_rescale (int32_int8_rescale) sử dụng đến 2 966 LUT logic do toàn bộ chuỗi biến đổi 5 bước: sign-extend, cộng round_bias, dịch phải số học, cộng zero_point, xén biên bão hòa; được hiện thực dưới dạng mạch tổ hợp song song cho 16 phần tử đồng thời mà không dùng bất kỳ phép nhân hay chia phần cứng nào. 224 CARRY8 phản ánh chuỗi cộng 34-bit và logic xén biên cần carry-chain để đạt tốc độ tối đa.

Hai bộ đệm bram_A và bram_B chiếm lần lượt 8 BRAM Tile mỗi khối, tổng cộng 16 BRAM Tile chỉ cho hai bộ đệm ping-pong đầu vào của gemm_compute_core. Riêng bram_A sử dụng nhiều LUT hơn bram_B do cấu trúc 16 BRAM song song tạo ra mạch tổng hợp phức tạp hơn ở phía đọc.

Module post_acc_inst tiêu thụ 514 LUT dưới dạng LUTRAM và 604 LUT logic — phản ánh hai cấu phần chính: vùng nhớ tích lũy acc_ram được Vivado suy luận thành LUTRAM, và mạch cộng song song 16 chiều thực hiện phép tích lũy INT32 mỗi chu kỳ, đòi hỏi nhiều LUT adder và carry-chain.

Module skew_inst sử dụng tới 285 LUT dưới dạng LUTRAM, tương ứng với chuỗi thanh ghi dịch của 16 luồng dữ liệu và tín hiệu điều khiển, được Vivado tối ưu thành SRL (Shift Register LUT) thay vì dùng FF thông thường. Đây là lý do skew_inst hầu như không tiêu thụ FF trong khi vẫn thực hiện chức năng trễ nhiều chu kỳ.

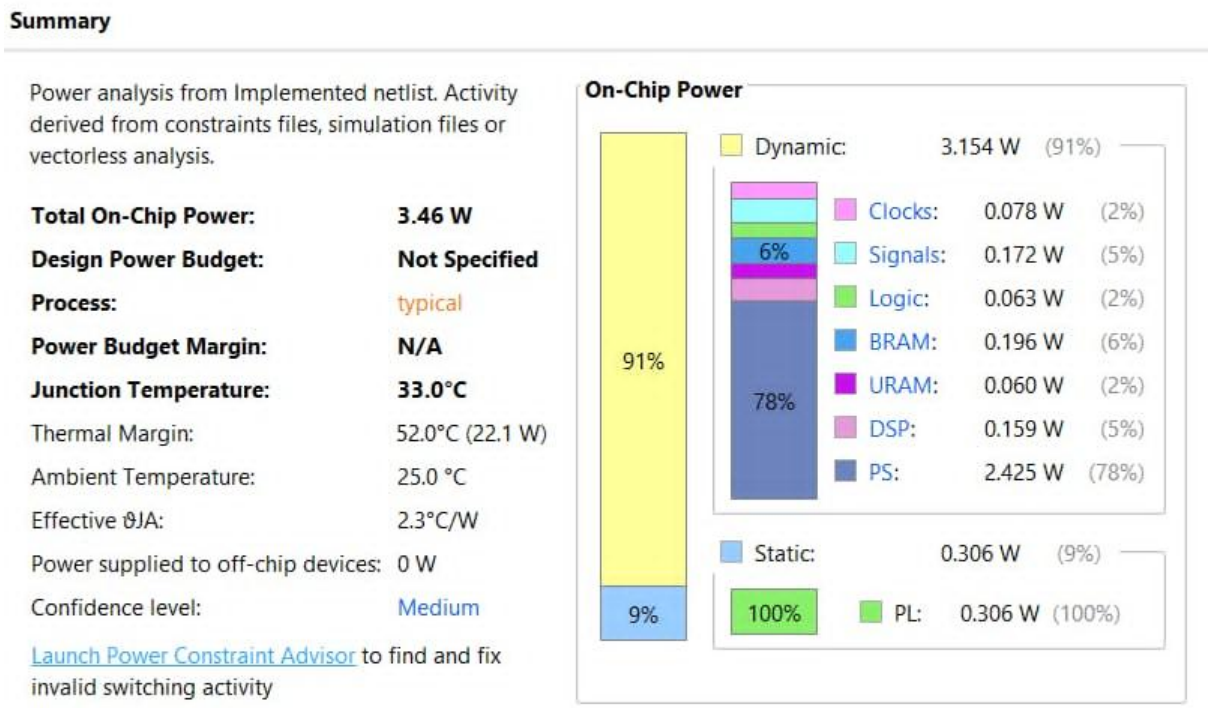
Module fifo_C_inst chiếm 14.5 BRAM Tile cho bộ đệm FIFO FWFT 512-bit, dùng làm vùng đệm giữa output_serializer và post_accumulator. Đây là BRAM đơn nhưng rộng và sâu, do đó chiếm nhiều tile hơn bram_A và bram_B.

Các module điều khiển ctrl_inst và serializer_inst chỉ chiếm tài nguyên rất nhỏ (dưới 70 LUT và dưới 530 FF), phù hợp với bản chất FSM thuần túy và logic tổ hợp đơn giản của chúng. serializer_inst có 521 FF so với 7 LUT do toàn bộ chức năng chủ yếu là đăng ký tín hiệu điều khiển và đệm dữ liệu drain, không đòi hỏi nhiều logic tổ hợp.

Nhìn tổng thể, hệ thống chỉ sử dụng khoảng 12.43% LUT logic, 12.45% FF và 20.51% DSP — cho thấy còn dư đáng kể tài nguyên logic và tính toán để mở rộng trong tương lai. Điểm đáng chú ý nhất là BRAM Tile đạt mức sử dụng 66.32%, cao hơn hẳn các loại tài nguyên khác. Điều này xuất phát từ yêu cầu lưu trữ on-chip của kiến trúc L2 tiling: GBUFA, GBUFB, bộ đệm ping-pong đầu vào, và FIFO C đều cùng sử dụng BRAM. Nếu cần mở rộng quy mô, BRAM sẽ là tài nguyên cần được cân nhắc ưu tiên trước tiên. Trong khi đó, 8 UltraRAM đang được sử dụng toàn bộ cho C_ACCUM — vùng tích lũy kết quả INT32 trung gian — chiếm 12.5% trong tổng số 64 URAM khả dụng, còn nhiều dư địa cho các ứng dụng cần bộ nhớ on-chip dung lượng lớn.

3.2.3. Công suất tiêu thụ (Power)

Công suất tiêu thụ được ước lượng bằng công cụ Vivado Power Analysis dựa trên netlist sau Place & Route kết hợp với thông tin hoạt động của tín hiệu. Kết quả phân tích Total On-Chip Power được thể hiện trong Hình 18 bên dưới.



Hình 18. Kết quả phân tích Total On-chip Power sau Implementation

Tổng công suất tiêu thụ trên chip đạt P_total = 3.46 W, trong đó công suất động P_dynamic = 3.154 W và công suất tĩnh P_static = 0.306 W. Tỷ lệ công suất động chiếm phần lớn này hoàn toàn phù hợp với đặc trưng của FPGA đang chạy tải tính toán nặng — phần lớn năng lượng đến từ switching activity của các khối DSP, BRAM và mạch clock phân phối.

Phân tích thành phần công suất động cho thấy phần PS tiêu thụ 2.425 W, chiếm khoảng 78% tổng công suất toàn chip. Trong khi đó, vùng PL chứa bộ tăng tốc GEMM chỉ tiêu thụ phần còn lại mặc dù đang thực hiện đồng thời 256 phép nhân cộng INT8 mỗi chu kỳ clock. Cụ thể, BRAM tiêu thụ 0.196 W, DSP tiêu thụ 0.159 W, clock network 0.078 W, switching activity của các tín hiệu 0.172 W, và logic tổ hợp 0.063 W.

Mức tiêu thụ tương đối thấp của PL được giải thích bởi một số đặc điểm kiến trúc. Việc ánh xạ phép nhân lên DSP48E2 cứng thay vì xây dựng bằng LUT giúp giảm đáng kể switching activity trên fabric logic. Kiến trúc Output-Stationary giữ giá trị cộng dồn cố định bên trong từng PE, hạn chế số lần truy cập bộ nhớ ngoài — vốn là nguồn tiêu thụ năng lượng lớn nhất trong các hệ thống AI accelerator. Cơ chế ping-pong buffer và pipeline dữ liệu liên tục cũng giúp giảm trạng thái stall, từ đó hạn chế các chu kỳ clock trống không có ích tiêu thụ năng lượng không cần thiết.

Về nhiệt độ, Vivado báo cáo nhiệt độ junction đạt 33 °C với thermal margin 52 °C so với giới hạn vận hành tối đa. Biên an toàn nhiệt lớn này cho thấy hệ thống vẫn còn khả năng hoạt động ổn định kể cả khi tăng tần số hoặc tải tính toán, không cần giải pháp tản nhiệt bổ sung trong điều kiện vận hành bình thường.

Tổng hợp lại, bộ tăng tốc GEMM đáp ứng đầy đủ cả ba tiêu chí PPA: đạt timing closure không vi phạm tại 200 MHz; sử dụng tài nguyên hiệu quả với mức chiếm dụng logic và DSP ở mức vừa phải, còn dư nhiều không gian để mở rộng; và duy trì mức tiêu thụ công suất PL thấp nhờ kiến trúc phần cứng được tối ưu phù hợp với đặc trưng của phép nhân ma trận INT8 dày đặc.

Chương IV. THỰC NGHIỆM HỆ THỐNG TRÊN FPGA.

1. Trích xuất ma trận đầu vào và đánh giá dữ liệu lượng tử hóa.

Để phục vụ cho việc tối ưu hóa phần cứng, các ma trận đầu vào (bao gồm ma trận Kích hoạt - Activation A và ma trận Trọng số - Weight B) được lượng tử hóa đối xứng sang cấu trúc INT8 dựa trên phương pháp AbsMax. Tập dữ liệu trích xuất để kiểm thử bao gồm: cặp ma trận kích thước 512 x 512 từ lớp q_proj, cùng ma trận kích thước 5120 x 1280 và 1280 x 5120 từ lớp fc1.

```
================================================================================
                BÁO CÁO KÍCH THƯỚC DỮ LIỆU - LỚP Q_PROJ (ATTENTION)
================================================================================
1. DỮ LIỆU THÔ TRÍCH XUẤT TỪ PYTORCH HOOK:
   - Activation gốc (act_raw) : [1500, 1280] (Thay đổi theo độ dài âm thanh)
--------------------------------------------------------------------------------
2. DỮ LIỆU ĐÃ ĐỊNH HÌNH CHO PHẦN CỨNG TĨNH (FP32):
   - Ma trận A (act_fp32)     : [512, 512]          | Mong đợi: [512, 512]
   - Ma trận B (weight_fp32)  : [512, 512]          | Mong đợi: [512, 512]
   - Ma trận C (Ground Truth) : [512, 512]          | Mong đợi: [512, 512]
--------------------------------------------------------------------------------
3. DỮ LIỆU SAU LƯỢNG TỬ HÓA - SẴN SÀNG XUẤT FILE BIN (INT8):
   - Ma trận A (act_int8)     : [512, 512]          | Kiểu: torch.int8
   - Ma trận B (weight_int8)  : [512, 512]          | Kiểu: torch.int8
================================================================================

================================================================================
                BÁO CÁO KÍCH THƯỚC DỮ LIỆU - LỚP FC1 (UPSAMPLING)
================================================================================
1. DỮ LIỆU THÔ TRÍCH XUẤT TỪ PYTORCH HOOK:
   - Activation gốc (act_raw) : [1500, 1280] (Thay đổi theo độ dài âm thanh)
--------------------------------------------------------------------------------
2. DỮ LIỆU ĐÃ ĐỊNH HÌNH CHO PHẦN CỨNG TĨNH (FP32):
   - Ma trận A (act_fp32)     : [5120, 1280]        | Mong đợi: [5120, 1280]
   - Ma trận B (weight_fp32)  : [1280, 5120]        | Mong đợi: [1280, 5120]
   - Ma trận C (Ground Truth) : [5120, 5120]        | Mong đợi: [5120, 5120]
--------------------------------------------------------------------------------
3. DỮ LIỆU SAU LƯỢNG TỬ HÓA - SẴN SÀNG XUẤT FILE BIN (INT8):
   - Ma trận A (act_int8)     : [5120, 1280]        | Kiểu: torch.int8
   - Ma trận B (weight_int8)  : [1280, 5120]        | Kiểu: torch.int8
================================================================================

✅ TẤT CẢ MA TRẬN TỪ 2 LỚP ĐỀU ĐÃ VÀO ĐÚNG KÍCH THƯỚC CHUẨN THIẾT KẾ PHẦN CỨNG!
```

Hình 19. Kiểm tra kích thước ma trận đã trích xuất

Chất lượng của quá trình lượng tử hóa từ định dạng FP32 sang INT8 được đánh giá thông qua các chỉ số đo lường chuẩn công nghiệp. Kết quả đối chiếu được thể hiện chi tiết tại Hình 20.

Các Chỉ Số Đo Lường	Q_PROJ Act	Q_PROJ Wgt	FC1 Act	FC1 Wgt	Chuẩn Lý Thuyết
Độ tương đồng Cosine (Hướng)	0.999638	0.997565	1.000570	0.999047	> 0.990000
Tỷ số Tín hiệu/Nhiễu (SQNR)	31.37 dB	23.10 dB	34.39 dB	24.74 dB	> 15.00 dB
Sai số tuyệt đối TB (MAE)	0.1614 LSB	0.2406 LSB	0.0684 LSB	0.2449 LSB	< 0.5000 LSB
Sai số bình phương gốc (RMSE)	0.2299 LSB	0.2812 LSB	0.1508 LSB	0.2854 LSB	Thấp (<0.6)
Mật độ phần tử bằng 0 (Sparsity)	43.24%	31.30%	73.83%	20.19%	Tự do
Dải giá trị thực tế sau Mapping	[-127, 59]	[-105, 127]	[-74, 127]	[-127, 108]	Trong [-128, 127]

Hình 20. Kết quả đối chiếu chất lượng lượng tử hóa đầu vào

Sai số toán học: Sai số tuyệt đối trung bình (MAE) của cả hai ma trận đều < 0.5 LSB, nằm hoàn hảo trong lưới số nguyên.

Độ tương đồng Cosine: Đạt mức cực cao (> 0.99), bảo toàn trọn vẹn hướng của các không gian vector nhúng.

Tỷ số SQNR: Vượt mốc 15 dB, khẳng định thông tin cốt lõi không bị méo.

Đánh giá trên phương diện lý thuyết, quá trình lượng tử hóa của các cặp ma trận đầu vào đã đáp ứng tốt các tiêu chuẩn kỹ thuật khắt khe. Dữ liệu sau lượng tử hóa duy trì được tính toàn vẹn cao, giúp giảm thiểu tối đa nguy cơ suy giảm độ chính xác (accuracy drop) khi thực thi mô hình Qwen2-Audio. Dữ liệu này đã hoàn toàn sẵn sàng để được nạp và triển khai trực tiếp trên các hệ thống phần cứng mạch RTL / FPGA.

2. Triển khai phần cứng lên KV260

Hệ thống được triển khai trên board AMD Kria KV260 chạy Debian Linux nhúng, giao tiếp với máy tính host qua SSH. Bitstream được nạp vào vùng PL qua FPGA Manager của Linux: chép file .bit vào /lib/firmware/ và kích hoạt qua /sys/class/fpga_manager/fpga0/firmware. Trạng thái hoạt động được xác nhận qua file state, hệ thống chỉ tiếp tục khi hiển thị operating.

3. Phần mềm điều khiển

Phần mềm điều khiển được viết bằng C, biên dịch trực tiếp trên KV260 bằng GCC với cờ tối ưu -O2. Chương trình truy cập phần cứng thông qua /dev/mem với quyền root, sử dụng mmap để ánh xạ không gian địa chỉ vật lý vào không gian người dùng.

3.1. Ánh xạ bộ nhớ vật lý

Chương trình mmap bốn vùng địa chỉ vật lý vào không gian người dùng:

Vùng	Địa chỉ vật lý	Kích thước	Mục đích
gemm_regs	0xA0000000	4 KB	12 thanh ghi AXI4-Lite: cấu hình + điều khiển
v_ptr_A / v_A	0x1E000000 / 0x63800000	1 MB / 8 MB	DDR buffer cho ma trận A (INT8)
v_ptr_B / v_B	0x1E100000 / 0x64000000	1 MB / 8 MB	DDR buffer cho ma trận B (INT8)
v_ptr_C / v_C	0x1E200000 / 0x64800000	1 MB / 32 MB	DDR buffer cho ma trận C kết quả (INT8)

Bảng 9. Ánh xạ bộ nhớ vật lý trong chương trình điều khiển

Địa chỉ DDR cho code C 5120×1280×5120 được chọn tại vùng 0x63800000–0x777FFFFF — vùng reserved không bị Linux kernel sử dụng, xác định bằng cách phân tích /proc/iomem trên board. Trước khi sử dụng, chương trình thực hiện pre-check:

mmap từng vùng, ghi 4 KB mẫu, đọc lại kiểm tra rồi giải phóng, đảm bảo vùng DDR thực sự khả dụng trước khi bắt đầu benchmark.

3.2. Cấu hình thanh ghi và khởi động

Trước mỗi lần thực thi, phần mềm ghi 11 tham số vào khối thanh ghi AXI4-Lite:

Thanh ghi	Tham số	Giá trị (512×512 / 5120×1280×5120)
REG_M_TOTAL [0]	Số hàng ma trận A	512 / 5120
REG_N_TOTAL [1]	Số cột ma trận B	512 / 5120
REG_K_TOTAL [2]	Chiều sâu K	512 / 1280
REG_K_DIM [3]	Chiều sâu K mỗi tile	32 / 32
REG_NUM_KT [4]	Số K-tile mỗi K-block	8 (256/32)
REG_BASE_A [5]	Địa chỉ DDR ma trận A	0x1E000000 / 0x63800000
REG_BASE_B [6]	Địa chỉ DDR ma trận B	0x1E100000 / 0x64000000
REG_BASE_C [7]	Địa chỉ DDR ma trận C	0x1E200000 / 0x64800000
REG_N_STRIDE[8]	Stride cột của C	512 / 5120
REG_SCALE_SH[9]	Hệ số dịch bit requantize	8
REG_ZERO_PT [10]	Zero-point bù lượng tử	0

Bảng 10. Danh sách thanh ghi cấu hình AXI4-Lite

Sau khi ghi xong toàn bộ tham số, phần mềm phát lệnh memory barrier __sync_synchronize() để đảm bảo tất cả AXI write được flush trước khi ghi bit start. Bit start (bit 0 của REG_CTRL_STAT) chỉ được ghi một lần duy nhất, tạo xung kích hoạt một chu kỳ cho phần cứng.

3.3. Cơ chế polling và đồng bộ hóa

Sau khi phát bit start, phần mềm vào vòng lặp polling liên tục đọc REG_CTRL_STAT và chờ bit busy hạ về 0. Thiết kế này đơn giản nhưng an toàn: phần cứng chỉ hạ cờ busy sau khi nhận đủ xác nhận S2MM từ DataMover, tức là toàn bộ kết quả đã được ghi vào DDR. Sau khi vòng lặp thoát, một memory barrier bổ sung đảm bảo CPU không đọc dữ liệu cũ từ cache trước khi truy cập kết quả.

Thanh ghi REG_CTRL_STAT có layout: [bit 2] done_sticky — [bit 1] busy — [bit 0] start. Cờ done_sticky là read-clear: phần cứng tự xóa ngay sau lần đọc đầu tiên, do đó không đọc lại lần thứ hai để kiểm tra. Trạng thái hoàn thành được xác nhận qua việc bit busy đã về 0 kết hợp với bit done_sticky trong lần đọc cuối của vòng polling.

3.4. Mô hình tham chiếu phần mềm

Song song với việc kích hoạt phần cứng, chương trình thực hiện phép nhân ma trận thuần CPU để làm chuẩn so sánh. Mô hình tham chiếu mô phỏng chính xác chuỗi xử lý của phần cứng ở mức bit:

- Tích lũy tích từng phần theo INT32.
- Cộng round_bias = 1 << (SCALE_SHIFT - 1) để làm tròn về giá trị gần nhất.
- Dịch phải số học SCALE_SHIFT bit (tương đương chia cho 2^{SCALE_SHIFT}).
- Cộng zero_point và xén biên bão hòa về miền [-128, 127].

Với bài kiểm thử 5120×1280×5120, để tránh cache miss do truy cập cột của B, phần mềm tạo bản chuyển vị B_T[n][k] = B[k][n] trước khi tính toán, biến vòng lặp trong (inner loop) thành truy cập tuần tự — tối ưu L1 cache trên ARM Cortex-A53. Kết quả CPU được đối chiếu từng phần tử với kết quả NPU; tỷ lệ khớp chính xác 100% là điều kiện xác nhận thiết kế hoạt động đúng.

4. Kết quả thực nghiệm

3.1. Kiểm thử chức năng — GEMM 512×512×512

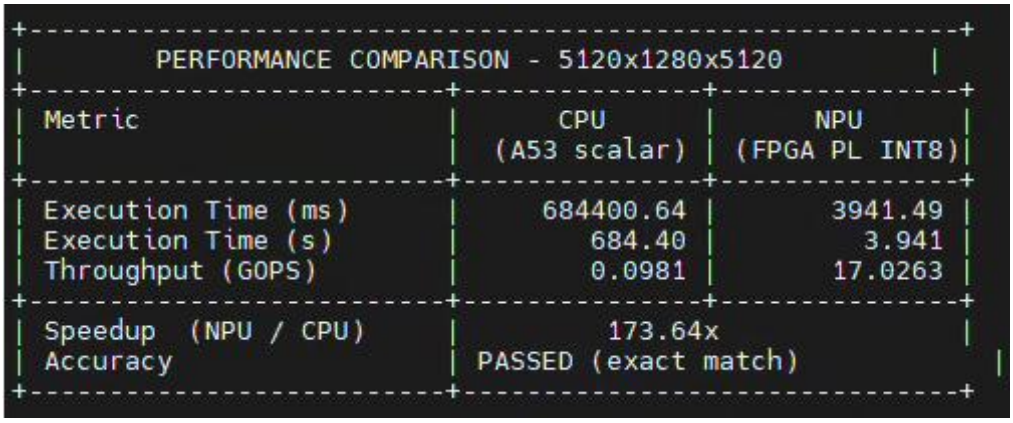
Bài kiểm thử 512×512×512 sử dụng K_DIM = 32, NUM_K_TILES = 8 (K_BLK = 256), SCALE_SHIFT = 8, ZERO_POINT = 0. Kết quả: ACCURACY CHECK PASSED — toàn bộ 262 144 phần tử của ma trận C khớp chính xác với mô hình tham chiếu CPU. Kết quả được thể hiện trong Hình 21.

```
+-------------------------------------------------------------+
|       PERFORMANCE COMPARISON - 512x512x512                  |
+--------------------------+---------------+----------------+
| Metric                   |      CPU      |      NPU       |
|                          |  (A53 scalar) | (FPGA PL INT8) |
+--------------------------+---------------+----------------+
| Execution Time (ms)      |      2737.493 |         18.679 |
| Execution Time (s)       |         2.737 |          0.019 |
|  -> Transfer In  (ms)    |             - |          0.533 |
|  -> Compute      (ms)    |             - |         16.739 |
|  -> Transfer Out (ms)    |             - |          1.407 |
| Throughput (GOPS)        |        0.0981 |        16.0366 |
+--------------------------+---------------+----------------+
| Speedup (Compute/CPU)    |         163.54x               |
| Speedup (E2E / CPU)      |         146.55x               |
| Accuracy                 | PASSED (exact match)          |
+--------------------------+-------------------------------+
```

Hình 21. Kết quả xử lý 512×512×512 trên KV260

3.2. Benchmark hiệu năng — GEMM 5120×1280×5120

Bài benchmark 5120×1280×5120 tương ứng với workload phép nhân ma trận trong các lớp Attention và Feed-Forward của mô hình Transformer. Tổng số phép tính đạt 2 × 5120 × 1280 × 5120 ≈ 6.71 × 10^10 ops. Hệ thống được cấu hình với K_DIM = 32, K_BLK = 256, NUM_K_TILES = 8. Kết quả được thể hiện trong Hình 22.



PERFORMANCE COMPARISON - 5120x1280x5120		
Metric	CPU (A53 scalar)	NPU (FPGA PL INT8)
Execution Time (ms)	684400.64	3941.49
Execution Time (s)	684.40	3.941
Throughput (GOPS)	0.0981	17.0263
Speedup (NPU / CPU)	173.64x	
Accuracy	PASSED (exact match)	

Hình 22. Kết quả benchmark 5120×1280×5120 trên KV260

Kết quả accuracy check: PASSED với 0 mismatch trên toàn bộ 5120×5120 = 26 214 400 phần tử, xác nhận tính đúng đắn của toàn bộ chuỗi xử lý từ L2 tiling, mảng tâm thu, tích lũy K-block, đến requantize INT32→INT8. Sau khi chạy xong, chương trình xuất kết quả ra file nhị phân output_C_cpu.bin và output_C_npu.bin để phục vụ kiểm định ở cấp độ framework cao hơn.

Chương V. ĐÁNH GIÁ HỆ THỐNG.

1. Đánh giá hiệu năng xử lí giữa PS và bộ tăng tốc PL trên KV260

Hiệu năng của hệ thống được đánh giá bằng cách so sánh thời gian thực thi phép nhân ma trận INT8 trên PS và PL. Các phép đo được thực hiện trực tiếp trên kit KV260 với hai bộ dữ liệu có kích thước 512×512×512 và 5120×1280×5120. Kết quả thực nghiệm được trình bày trong Hình 21 và Hình 22.

Đối với bài toán 512×512×512, thời gian thực thi trên CPU là 2737.493 ms, trong khi bộ tăng tốc hoàn thành sau 18.679 ms, trong đó thời gian tính toán của mảng xử lý chỉ chiếm 16.739 ms. Hệ thống đạt thông lượng 16.04 GOPS, cao hơn đáng kể so với 0.098 GOPS của CPU. Tốc độ xử lý của phần cứng nhanh hơn 163.54 lần so với CPU, hoặc 146.55 lần nếu tính cả thời gian truyền dữ liệu giữa PS và PL. Kết quả kiểm tra cho thấy đầu ra của bộ tăng tốc hoàn toàn trùng khớp với kết quả tham chiếu trên CPU.

Đối với bài toán 5120×1280×5120, CPU cần 684400.64 ms để hoàn thành phép nhân ma trận, trong khi bộ tăng tốc chỉ mất 3941.49 ms. Thông lượng đạt 17.03 GOPS, gần tương đương với bài toán kích thước nhỏ, cho thấy khả năng duy trì hiệu suất ổn định khi khối lượng tính toán tăng lên. Tốc độ xử lý của bộ tăng tốc nhanh hơn 173.64 lần so với CPU, đồng thời vẫn đảm bảo kết quả đầu ra khớp hoàn toàn với mô hình tham chiếu.

Từ hai kết quả thực nghiệm có thể thấy việc chuyển phép nhân ma trận từ PS sang PL mang lại hiệu quả tăng tốc rõ rệt. Kiến trúc Systolic Array kết hợp cơ chế truyền dữ liệu bằng AXI DataMover và xử lý INT8 đã khai thác tốt khả năng song song của FPGA, giúp duy trì thông lượng trên 16 GOPS đối với cả bài toán nhỏ và bài toán có kích thước lớn. Đồng thời, kết quả kiểm thử đều đạt PASS với chương trình tham chiếu, khẳng định tính đúng đắn và độ ổn định của toàn bộ hệ thống.

2. Đánh giá tính đúng đắn và hiệu năng với Python

Để kiểm tra và đánh giá toàn diện độ chính xác của bộ tăng tốc GEMM trên kit KV260, hệ thống được thẩm định qua hai giai đoạn độc lập: Xác thực chéo logic phần cứng với mô hình chuẩn Golden Model và Đo lường chất lượng nhiễu lượng tử theo tiêu chuẩn công nghiệp dành cho mô hình ngôn ngữ lớn.

2.1. Xác thực chéo logic phần cứng và độ chính xác Bit-True

Quá trình kiểm định sử dụng một mô hình Golden Model được xây dựng trên framework PyTorch để mô phỏng chính xác từng bước xử lý trong thiết kế. Mô hình này tái hiện trọn vẹn chuỗi pipeline xử lý số nguyên bao gồm: phép nhân tích lũy trên thanh ghi INT32, khâu cộng sai số làm tròn, toán tử dịch bit phải, và cuối cùng là mạch bão hòa dữ liệu để đưa ma trận kết quả về định dạng INT8.

Kết quả đối chiếu trực tiếp giữa file nhị phân đầu ra thực tế từ chip KV260 và kết quả mô phỏng của Golden Model được thể hiện ở Hình 23.

```
================================================================================================================
                BẢNG VÀNG XÁC THỰC PHẦN CỨNG: BIÊN DỊCH TOÁN HỌC PYTORCH VS ĐẦU RA MẠCH RTL NPU
================================================================================================================
| Phân Lớp Mục Tiêu  | Kích Thước Ma Trận      | Số Điểm Lệch Dữ Liệu        | Sai Số Max (LSB) | Trạng Thái RTL |
----------------------------------------------------------------------------------------------------------------
| Attention Q_PROJ   | 512x512 x 512x512       | 0 / 262,144                 | 0 LSB            | ✅ MATCH 100%  |
| FFN Upsample FC1   | 5120x1280 x 1280x5120   | 0 / 26,214,400              | 0 LSB            | ✅ MATCH 100%  |
================================================================================================================
```

Hình 23. Kết quả kiểm tra Bit-True

Kết quả này minh chứng hệ thống đạt trạng thái Đồng bộ hoàn hảo ở mức Bit-True. Điều này khẳng định logic điều khiển, các khối tính toán RTL trên FPGA và cấu trúc pipeline dịch bit/làm tròn trong thiết kế phần cứng hoàn toàn chính xác tuyệt đối về mặt toán học số nguyên so với framework chuẩn.

2.2. Phân tích sai số và chất lượng lượng tử hóa cấu trúc LLM

Bên cạnh việc xác thực logic số nguyên, chất lượng của thuật toán lượng tử hóa và mạch giải lượng tử ngược cũng được đánh giá bằng cách so sánh đầu ra của bộ tăng tốc với ma trận kết quả gốc FP32. Quá trình kiểm định được thực hiện chi tiết trên hai lớp mạng đặc trưng của mô hình Qwen2 là lớp chiếu q_proj (kích thước khối 512) và lớp mở rộng mạng FFN fc1 (kích thước khối 5120). Các chỉ số sai số thu được cụ thể như Hình 24.

```
================================================================================================================
Các Chỉ Số Đo Lường (C = A x B)     | Output Q_PROJ (512)  | Output FC1 (5120)    | Chuẩn LLM Lý Thuyết
----------------------------------------------------------------------------------------------------------------
Độ tương đồng Cosine (Hướng)        | 0.993575             | 0.998120             | > 0.990000
Tỷ số Tín hiệu/Nhiễu (SQNR)         | 18.88 dB             | 22.69 dB             | > 15.00 dB
Sai số tuyệt đối TB (MAE Requant)   | 0.036134 FP32        | 0.035945 FP32        | Nhỏ nhất có thể
Sai số bình phương gốc (RMSE)       | 0.043156 FP32        | 0.097420 FP32        | Thấp (< 1.0)
Dải giá trị INT8 thu được trên NPU  | [-19, 19]            | [-53, 110]           | Trong [-128, 127]
================================================================================================================
```

Hình 24. Kết quả đánh giá giải lượng tử kết quả GEMM

Từ kết quả thực nghiệm, chỉ số độ tương đồng Cosine của cả hai ma trận đầu ra đều vượt mốc tiêu chuẩn công nghiệp 0.99 (cụ thể đạt 0.993575 đối với lớp q_proj và 0.998120 đối với lớp fc1). Kết quả này chứng minh thiết kế đã bảo toàn nguyên vẹn đặc trưng không gian và hướng vector của các tầng ẩn trong mô hình LLM. Đồng thời, tỷ số tín hiệu trên nhiễu lượng tử (SQNR) đạt lần lượt 18.88 dB và 22.69 dB, vượt xa mốc lý thuyết yêu cầu (> 15.00 dB). Điều này khẳng định nhiễu biên độ sinh ra do quá trình ép kiểu từ FP32 xuống INT8 hoàn toàn nằm trong giới hạn hội tụ cho phép của cấu trúc Per-Tensor.

Bên cạnh đó, các chỉ số sai số tuyệt đối trung bình (MAE) sau khi giải lượng tử duy trì ở mức cực kỳ thấp (khoảng 0.036 cho cả hai lớp). Dải giá trị INT8 thu được trên thiết kế thực tế nằm an toàn trong khoảng [-128, 127] (chỉ dao động ở mức [-19, 19] với

q_proj và [-53, 110] với fc1), hoàn toàn không xảy ra hiện tượng tràn số bão hòa. Hệ thống phần cứng kiểm soát rất tốt lỗi nhiễu lượng tử, đảm bảo đầu ra của cả khối Attention và khối FFN bám sát không gian vector gốc mà không gây ra hiện tượng Accuracy Drop hay sụp đổ mô hình khi triển khai.

Thông qua bộ chỉ số thực nghiệm khắt khe, thiết kế bộ tăng tốc GEMM trên nền tảng FPGA nhúng không chỉ đảm bảo tính đúng đắn tuyệt đối trong khâu thực thi logic phần cứng, mà còn chứng minh khả năng tối ưu hóa toán học xuất sắc, đáp ứng đầy đủ các tiêu chuẩn kỹ thuật khắt khe của công nghiệp AI/LLM về độ chính xác tín hiệu.

Chương VI. KẾT LUẬN.

Đồ án đã nghiên cứu, thiết kế và hiện thực thành công bộ tăng tốc phần cứng cho phép nhân ma trận trên nền tảng KV260. Hệ thống được xây dựng theo mô hình đồng thiết kế PS–PL, trong đó bộ xử lý ARM Cortex-A53 đảm nhiệm việc cấu hình và điều khiển, còn vùng Programmable Logic thực hiện tính toán song song bằng kiến trúc Systolic Array. Thiết kế hỗ trợ dữ liệu INT8, tích hợp cơ chế phân khối ma trận bằng phần cứng, truyền dữ liệu qua AXI DataMover và giao tiếp AXI4-Stream, đáp ứng yêu cầu xử lý các bài toán GEMM có kích thước lớn.

Kết quả triển khai thực tế trên kit KV260 cho thấy hệ thống hoạt động ổn định và cho kết quả chính xác với cả hai bài toán kiểm thử 512×512×512 và 5120×1280×5120. Kết quả đầu ra của bộ tăng tốc trùng khớp hoàn toàn với chương trình tham chiếu trên CPU và Golden Model xây dựng bằng Python. Các chỉ số Cosine Similarity, SQNR, RMSE và MAE cũng cho thấy quá trình lượng tử hóa và tái chuẩn hóa vẫn bảo toàn tốt đặc trưng của dữ liệu, đáp ứng yêu cầu triển khai cho các mô hình AI sử dụng số nguyên INT8.

Về hiệu năng, bộ tăng tốc phần cứng đạt tốc độ xử lý vượt trội so với việc thực thi thuần trên ARM Cortex-A53. Đối với bài toán 512×512×512, hệ thống đạt tốc độ nhanh hơn khoảng 163 lần ở thời gian tính toán và 146 lần khi tính toàn bộ thời gian thực thi. Với bài toán 5120×1280×5120, tốc độ tăng lên khoảng 174 lần, cho thấy kiến trúc đề xuất có khả năng mở rộng tốt khi kích thước ma trận và khối lượng tính toán ngày càng lớn.

Trong tương lai, hệ thống có thể được tiếp tục phát triển theo hướng mở rộng kích thước mảng Systolic Array, tối ưu cơ chế điều phối dữ liệu nhằm khai thác tốt hơn băng thông DDR, đồng thời tích hợp trực tiếp với các framework suy luận AI như ONNX Runtime hoặc Vitis AI để xây dựng bộ gia tốc hoàn chỉnh cho các mô hình Transformer và Large Language Model trên nền tảng FPGA.

PHỤ LỤC

Source code thiết kế: GitHub

NGUỒN THAM KHẢO

[1] Qwen Team, "Qwen2 Technical Report," Alibaba Group, 2024. [Online]. Available: arXiv:2405.14458.

[2] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] Quang-Chung Tran, "Qwen2-Audio-Dialogue Fine-tuned Model," Hugging Face Repository, 2024. [Online]. Available: https://huggingface.co/tranquangchung/qwen2-audio-dialogue.

[4] X. Zhao *et al.*, "Linear Symmetric Quantization of Neural Networks for Low-precision Integer Hardware," in *International Conference on Learning Representations (ICLR)*, 2020.

[5] S. Williams *et al.*, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, 2009.

[6] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, 2017, pp. 1-12.

[7] B. Xu, A. Banerjee, and S. Gupta, "Hardware Acceleration for Neural Networks: A Comprehensive Survey," Arizona State University, 2024.

[8] J. Li *et al.*, "Hummingbird+: Advancing FPGA-based LLM Deployment from Research Prototype to Edge Product," in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2026.

[9] R. Kastner, J. Matai, and S. Neuendorffer, *Parallel Programming for FPGAs*, UC San Diego, 2018.

[10] Advanced Micro Devices (AMD), *Kria KV260 Vision AI Starter Kit User Guide (UG1089)*, 2023.

[11] A. Fengwei *et al.*, "A Component-Centric Perspective on Hardware Accelerators for LLMs," *IEEE Access*, vol. 12, 2024.

[12] G. E. Moon *et al.*, "Evaluating Spatial Accelerator Architectures with Tiled Matrix-Matrix Multiplication," *IEEE Transactions on Parallel and Distributed Systems*, 2021.

[13] T. Raja, "Systolic Array Data Flows for Efficient Matrix Multiplication in Deep Neural Networks," *arXiv preprint arXiv:2410.22595*, 2024.

[14] Advanced Micro Devices (AMD), *Block Memory Generator v8.4 Product Guide (PG058)*, 2021.

[15] Advanced Micro Devices (AMD), *AXI DataMover v5.1 LogiCORE IP Product Guide (PG022)*, v5.1, Jan. 2022. [Online]. Available: https://www.xilinx.com/support/documents/ip_documentation/axi_datamover/v5_1/pg022_axi_datamover.pdf

[16] Advanced Micro Devices (AMD), *AXI DMA LogiCORE IP Product Guide (PG021)*, 2025.

[17] Arm Limited, "*AMBA® AXI Protocol Specification*," ARM IHI 0022, Issue L, Aug. 27, 2025.

[18] Real Digital, "Introduction to AXI4-Lite Interface," Tech. Rep., 2025.

[19] ARM, *AMBA 4 AXI4-Stream Protocol Specification (Version 1.0)*, ARM Ltd., 2010.

[20] D. Baptista, L. Sousa, and F. Morgado-Dias, "Raising the Abstraction Level of a Deep Learning Design on FPGAs," *IEEE Access*, vol. 8, pp. 205151-205161, 2020, doi: 10.1109/ACCESS.2020.3036975